

**курс «Глубокое обучение»**

**Трансформер**

**Александр Дьяконов**

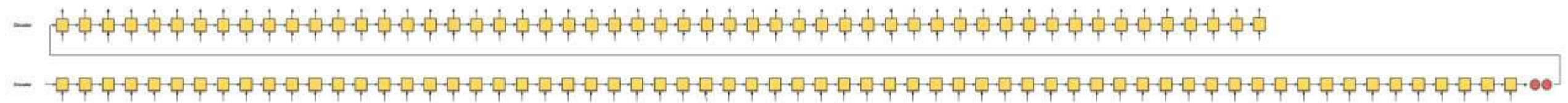


## План

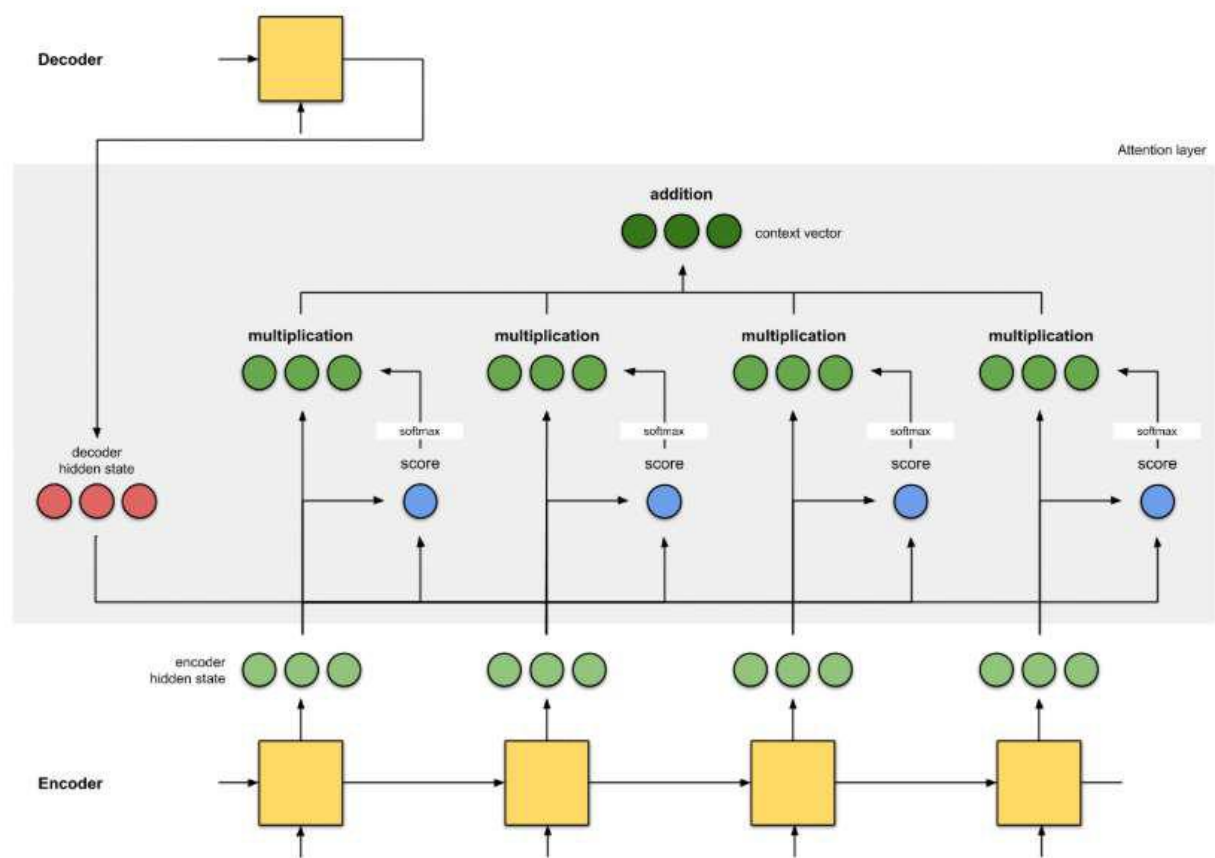
**attention / self- attention – матричная запись**  
**Transformer**  
**Особенности обучения трансформера**  
**BERT**

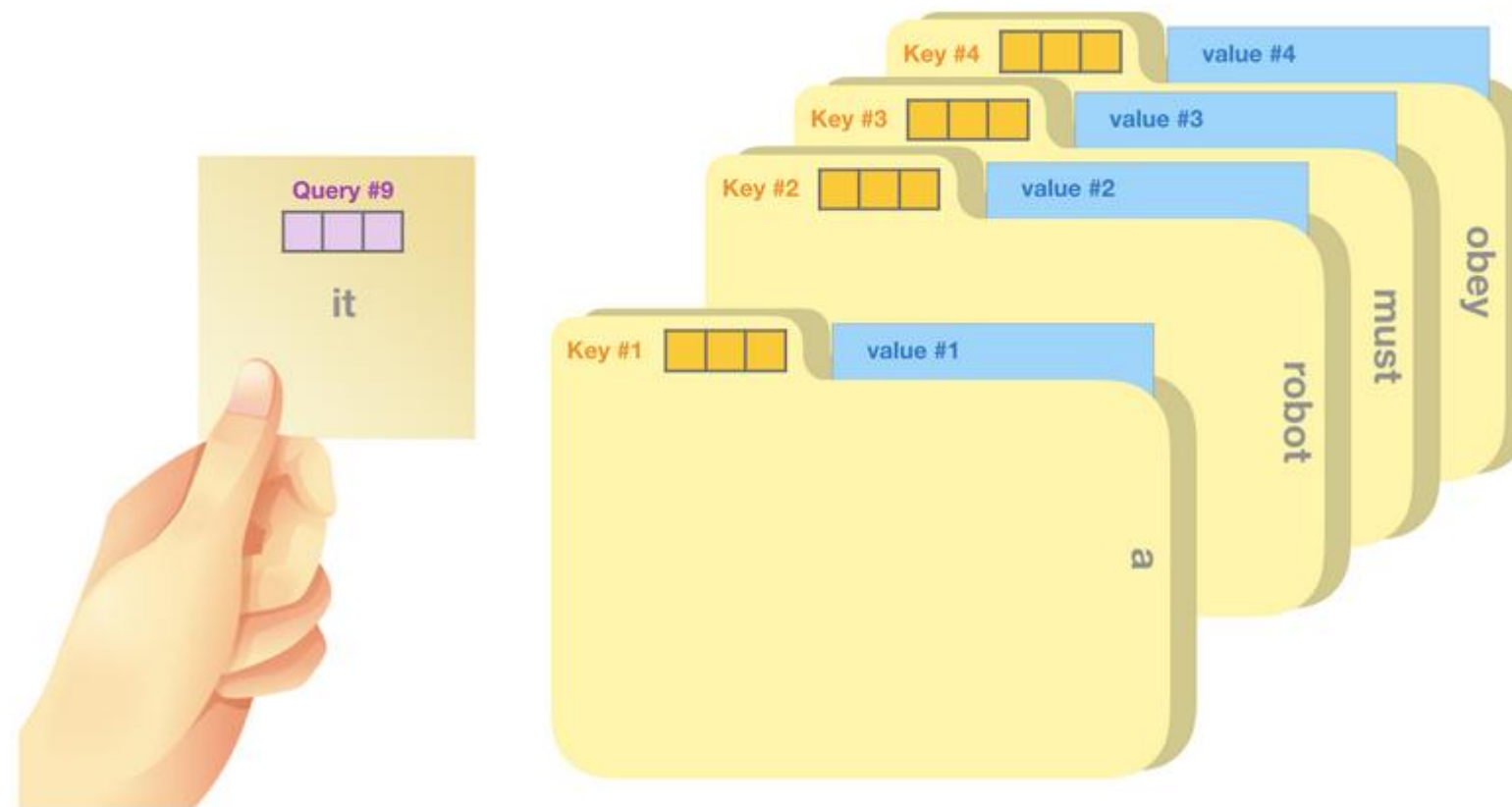
Рис. из <https://towardsdatascience.com/@remykarem>

# Напоминание: внимание seq2seq



## seq2seq + attention: Query, Key, Value



**attention / self-attention****Парадигма Query, Key, Value (запрос, ключ, значение)**

**attention / self-attention – матричная запись****идея «attention»...****values** –  $V_{d \times s}$ **keys** –  $K_{d \times s}$ **query** –  $q$ **размерность** –  $d$ **число примеров** –  $S$ **«self-attention»****в  $X$  перечислены объекты внимания**

$$V_{d \times s} = W_{d \times D}^V X_{D \times s}$$

$$K_{d \times s} = W_{d \times D}^K X_{D \times s}$$

$$Q_{d \times s} = W_{d \times D}^Q X_{D \times s}$$

**размерности  $K$  и  $V$  м.б. разные**

$$v = (V_{d \times s} \text{softmax}(K_{s \times d}^T q_{d \times 1})_{s \times 1})_{d \times 1}$$

**в матричной форме + нормировка**

$$(V \text{softmax}(\alpha K^T Q))_{d \times s}$$

$$\text{head}(X \mid W^V, W^K, W^Q) = W^V X \text{softmax}\left(\alpha (W^K X)^T W^Q X\right), \alpha = 1 / \sqrt{d}$$

**+ сделать несколько параллельных матриц**

$$W_{D \times 8d} \cdot \text{concat}[\text{head}(W_t^V, W_t^K, W_t^Q)_{d \times s}, \text{axis} = 0]_{t=1}^8$$

## Минутка кода: attention

```
def attention(query, key, value, mask=None, dropout=None):  
    "Compute 'Scaled Dot Product Attention'"  
    d_k = query.size(-1)  
    scores = torch.matmul(query, key.transpose(-2, -1)) / math.sqrt(d_k)  
    if mask is not None:  
        scores = scores.masked_fill(mask == 0, -1e9)  
    p_attn = F.softmax(scores, dim = -1)  
    if dropout is not None:  
        p_attn = dropout(p_attn)  
    return torch.matmul(p_attn, value), p_attn
```

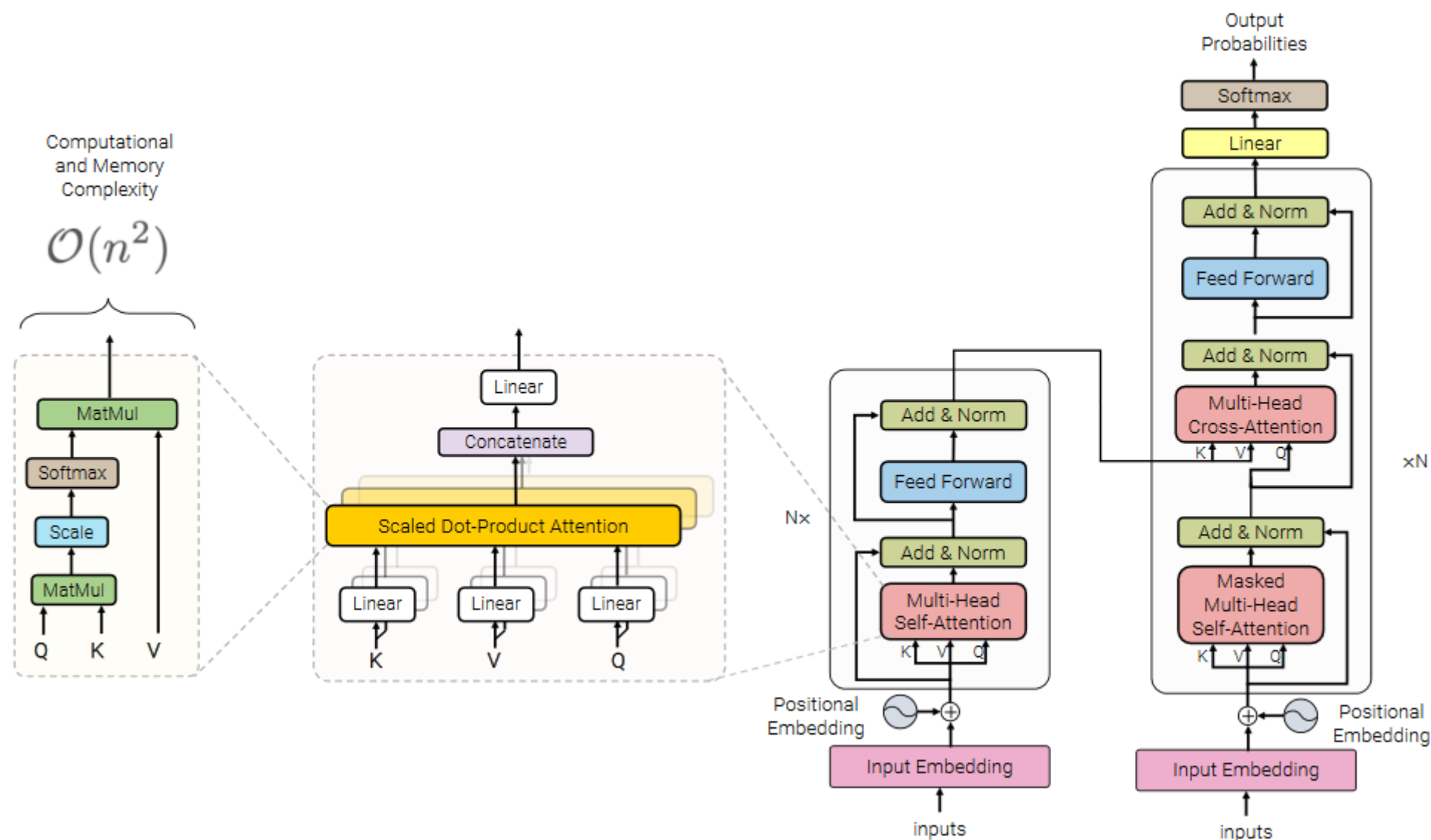
```
class MultiHeadedAttention(nn.Module):
    def __init__(self, h, d_model, dropout=0.1):
        "Take in model size and number of heads."
        super(MultiHeadedAttention, self).__init__()
        assert d_model % h == 0
        # We assume d_v always equals d_k
        self.d_k = d_model // h
        self.h = h
        self.linears = clones(nn.Linear(d_model, d_model), 4) # W_q, W_k, W_v, W
        self.attn = None
        self.dropout = nn.Dropout(p=dropout)

    def forward(self, query, key, value, mask=None):
        if mask is not None: # Same mask applied to all h heads.
            mask = mask.unsqueeze(1)
        nbatches = query.size(0)

        # Получить проекции, т.е. матрицы Q, K, V => h x d_k
        query, key, value = [l(x).view(nbatches, -1, self.h, self.d_k).transpose(1, 2)
                               for l, x in zip(self.linears, (query, key, value))]

        # само внимание.
        x, self.attn = attention(query, key, value, mask=mask, dropout=self.dropout)
        # конкатенация + линейность
        x = x.transpose(1, 2).contiguous().view(nbatches, -1, self.h * self.d_k)
        return self.linears[-1](x)
```

# Transformer: Основная идея «Parallelized Attention»

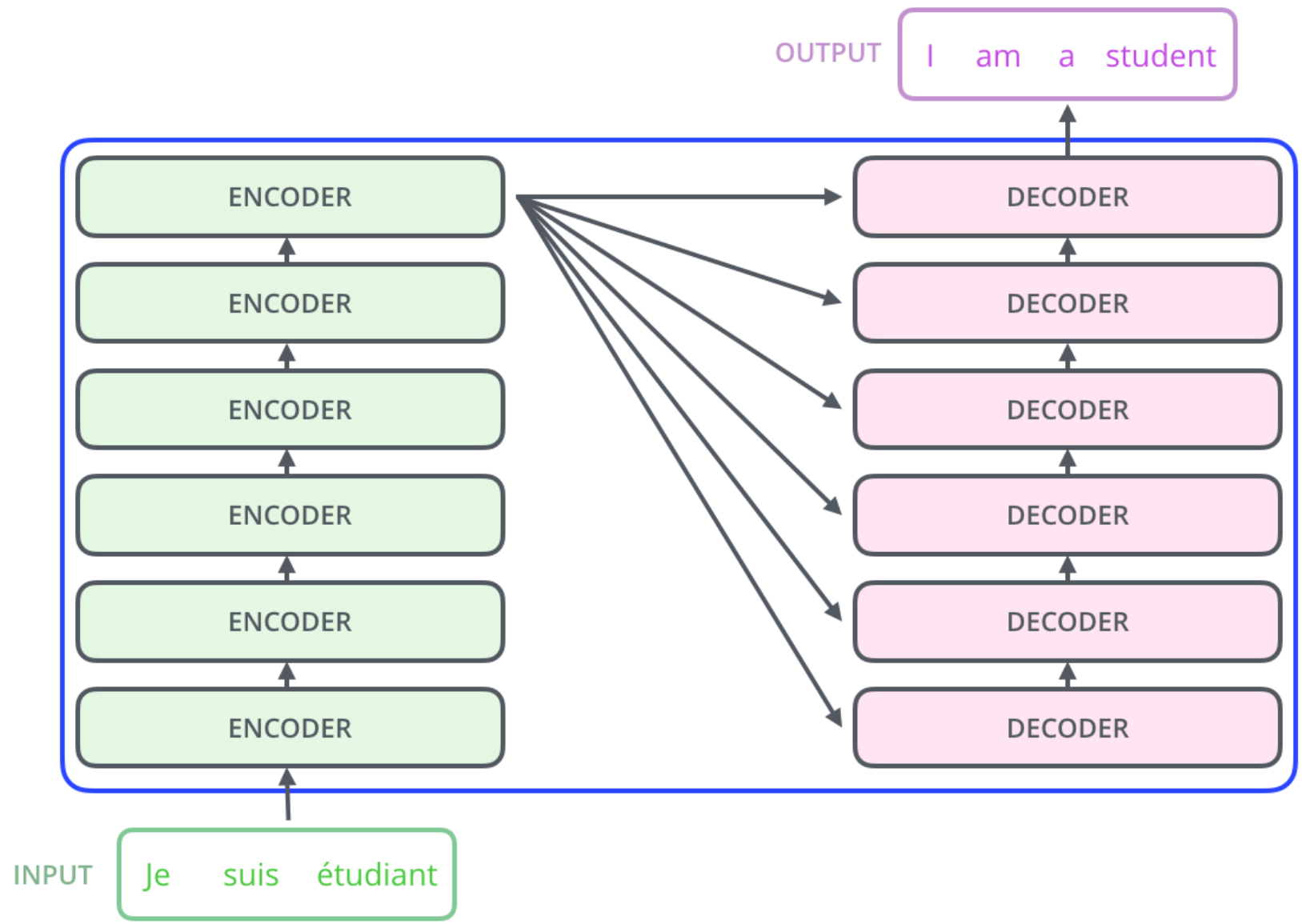


Vaswani et al. «Attention Is All You Need» <https://arxiv.org/abs/1706.03762>

Дальше картинки из <https://jalammar.github.io/illustrated-transformer/>

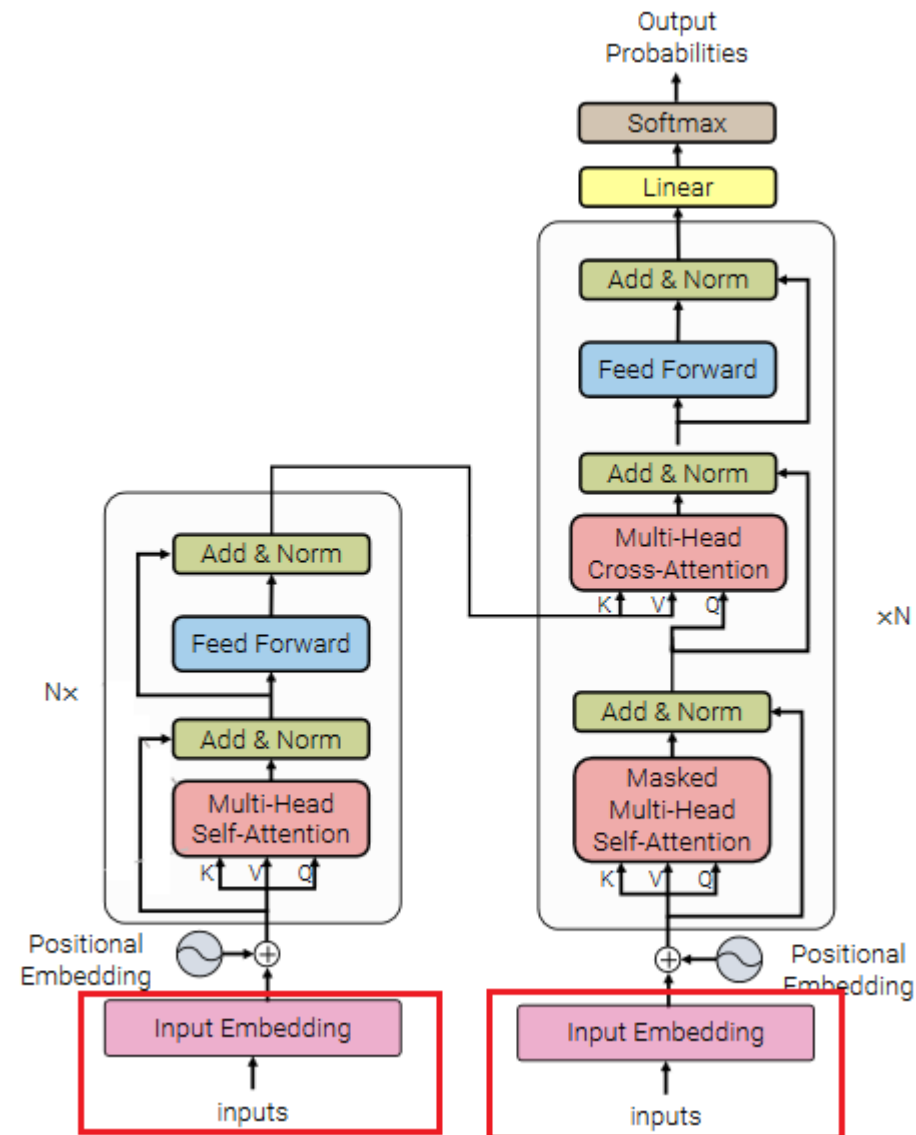


Transformer: общий вид

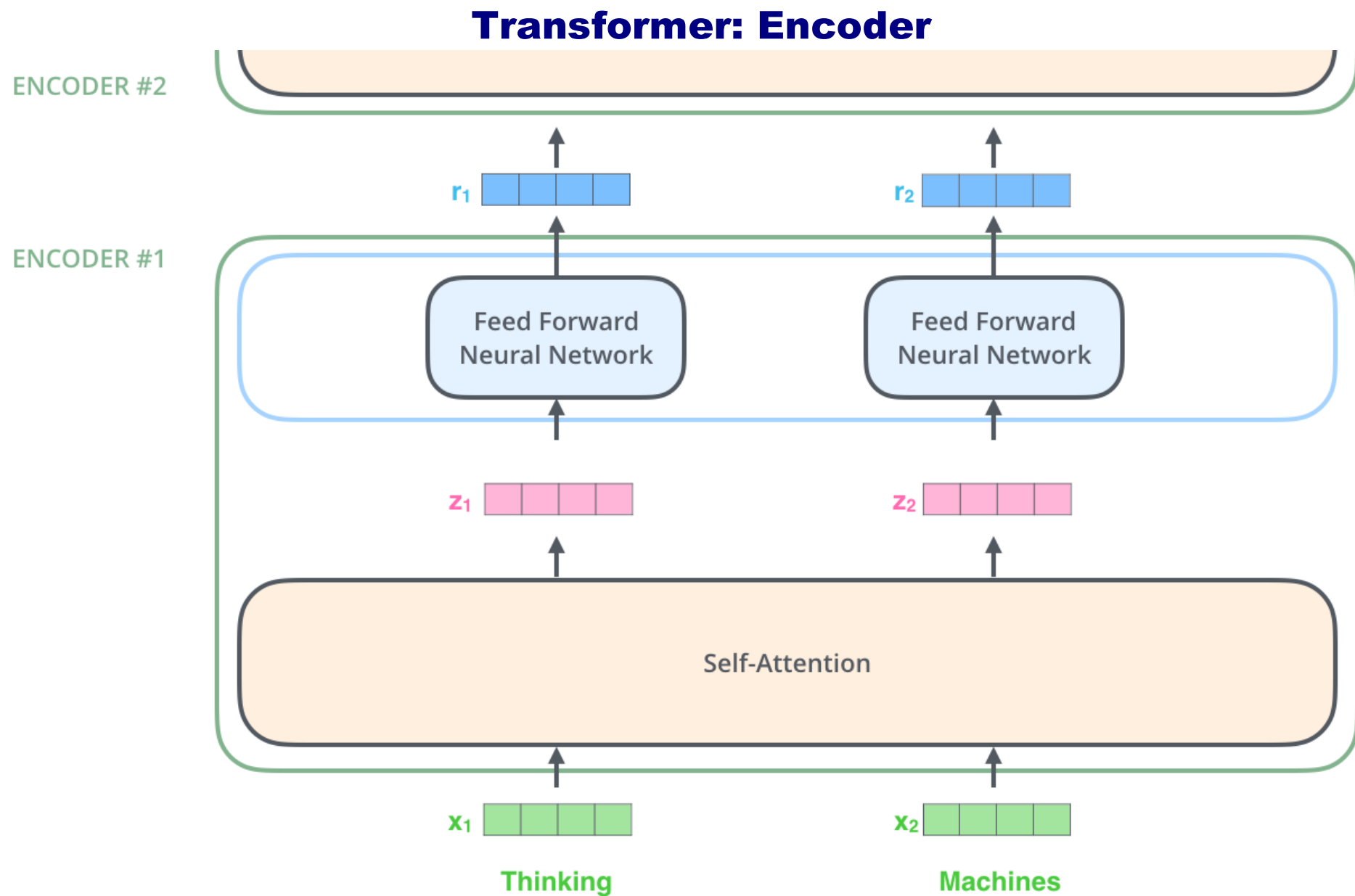


**опять схема «кодировщик» – «декодировщик» (6 + 6 слоёв)**  
**компоненты одинаковые по архитектуре, но веса разные**

## Transformer: Encoder



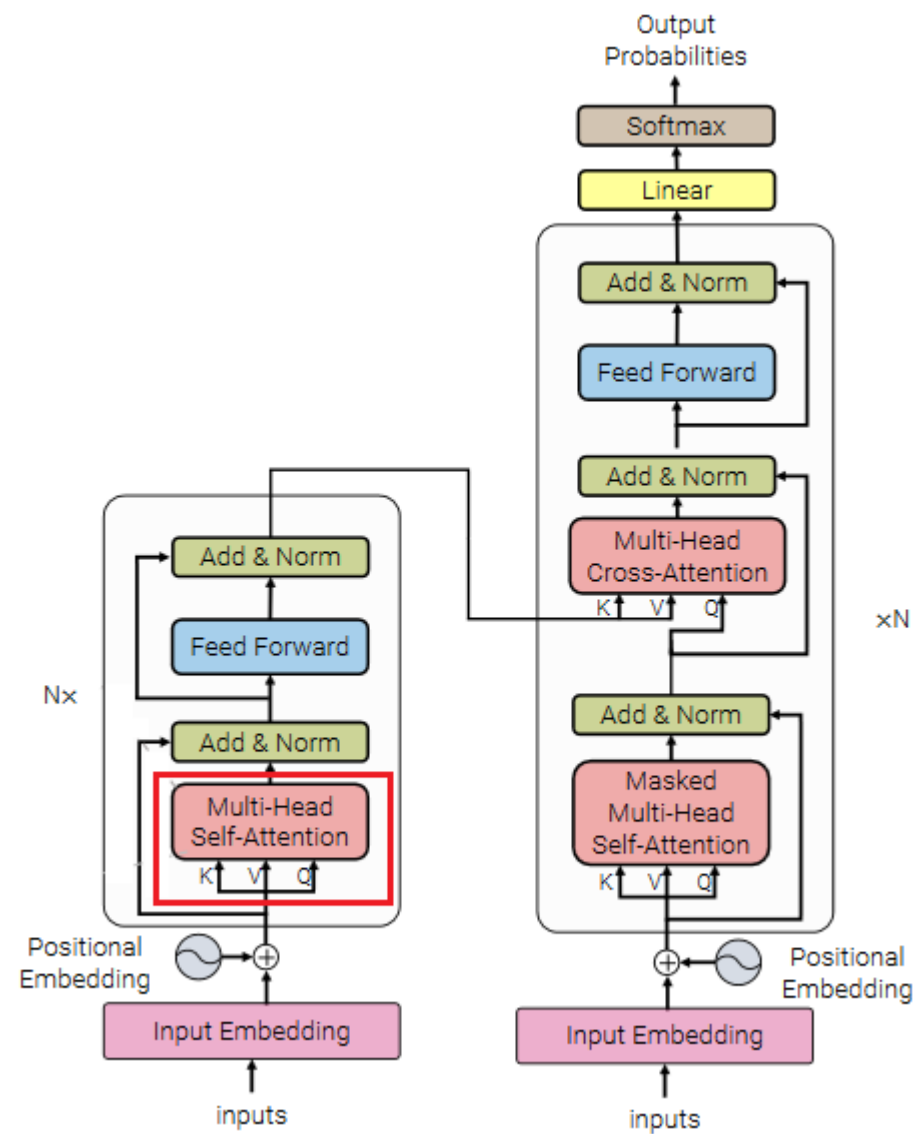
**на вход подаются обучаемые представления токенов (сначала делается токенизация)**



**слова подаём в виде представлений (embeddings)  $R^{512}$**

**их преобразуем в векторы такой же размерности (представление следующего уровня)**

# Концепция «Self-Attention»



механизм внимания будет менять представление с учётом контекста

# Концепция «Self-Attention»

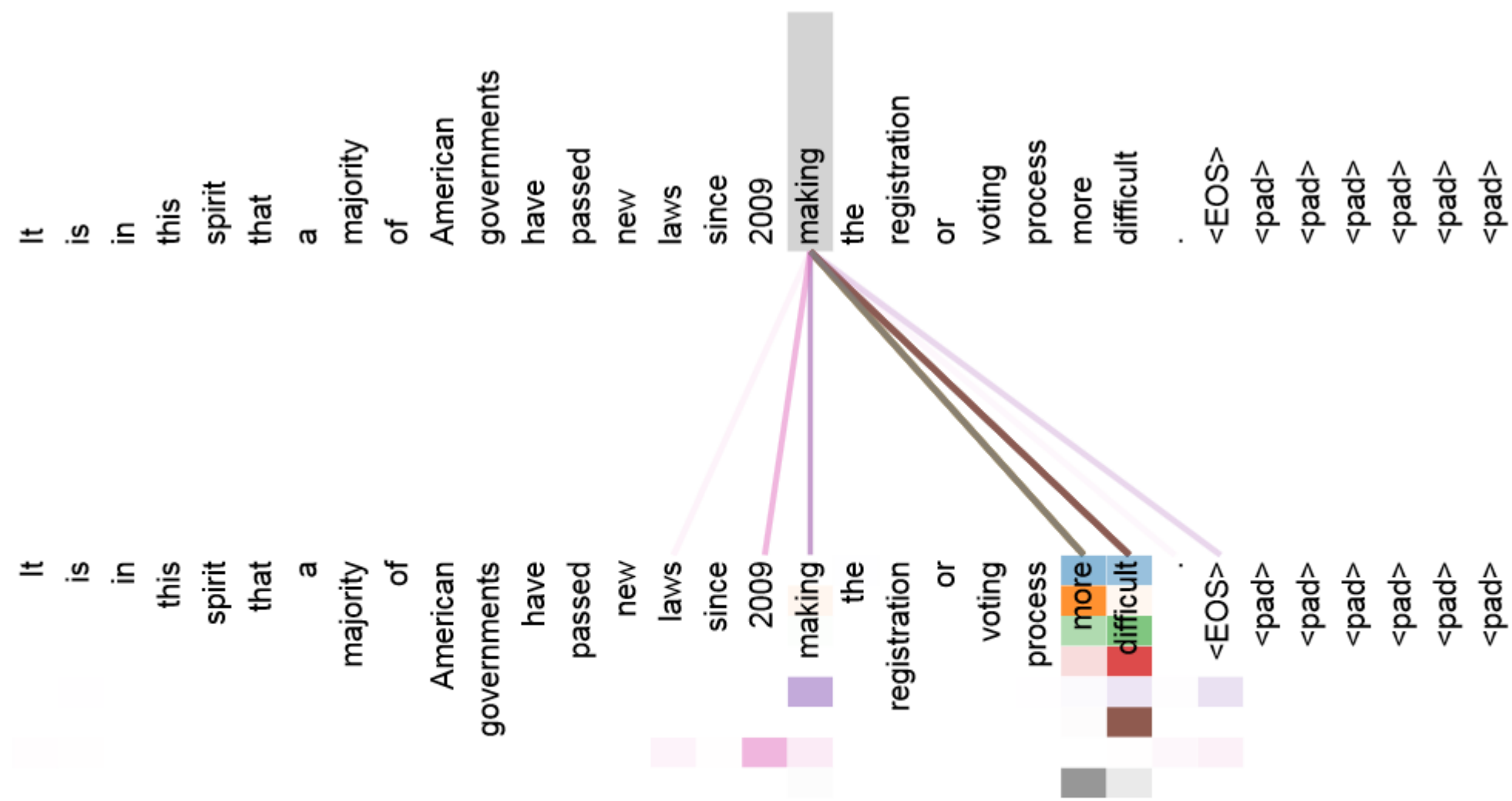
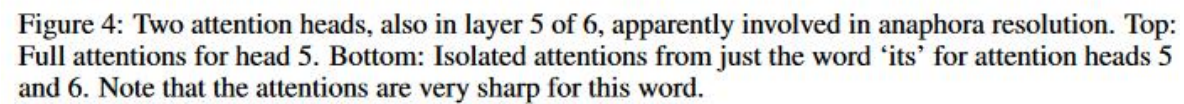


Figure 3: An example of the attention mechanism following long-distance dependencies in the encoder self-attention in layer 5 of 6. Many of the attention heads attend to a distant dependency of the verb ‘making’, completing the phrase ‘making...more difficult’. Attentions here shown only for the word ‘making’. Different colors represent different heads. Best viewed in color.



# Концепция «Self-Attention»: есть специальные визуализаторы внимания

Select model

bert-base-cased

▼

Input Sentence

Whoever reads this is an idiot.

Update

Filters

Hide Special Tokens

☐

Show top 50% of att:

Layer

1

2

3

4

5

6

7

8

9

10

11

12

Selected heads:

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12

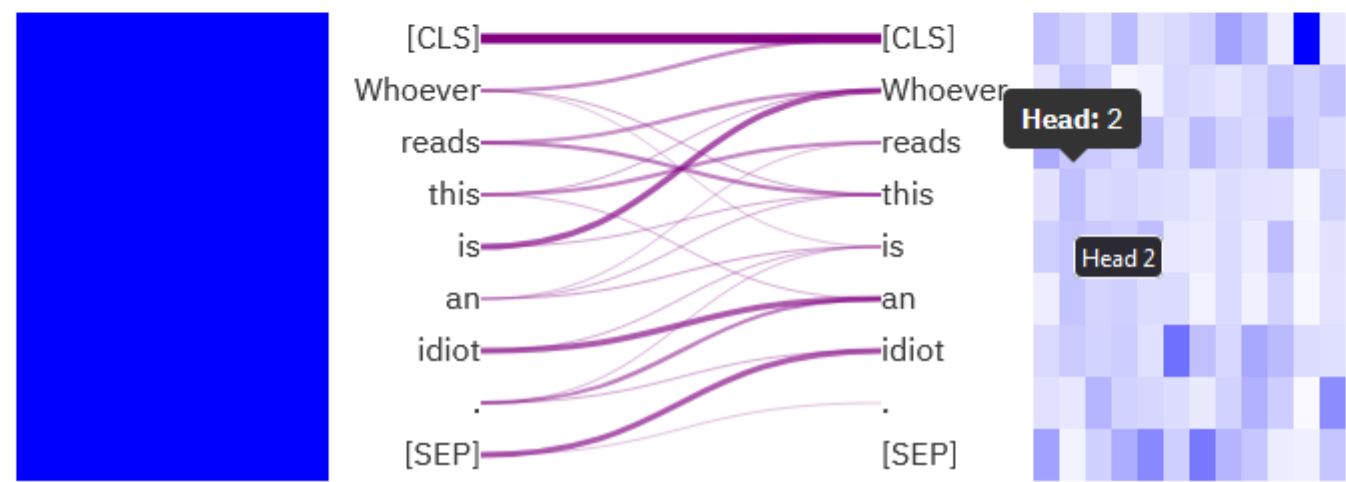
Select all heads

Unselect all heads

You *focus* on one token by **click**. For bidirectional models, you can *mask* any token by **double click**.

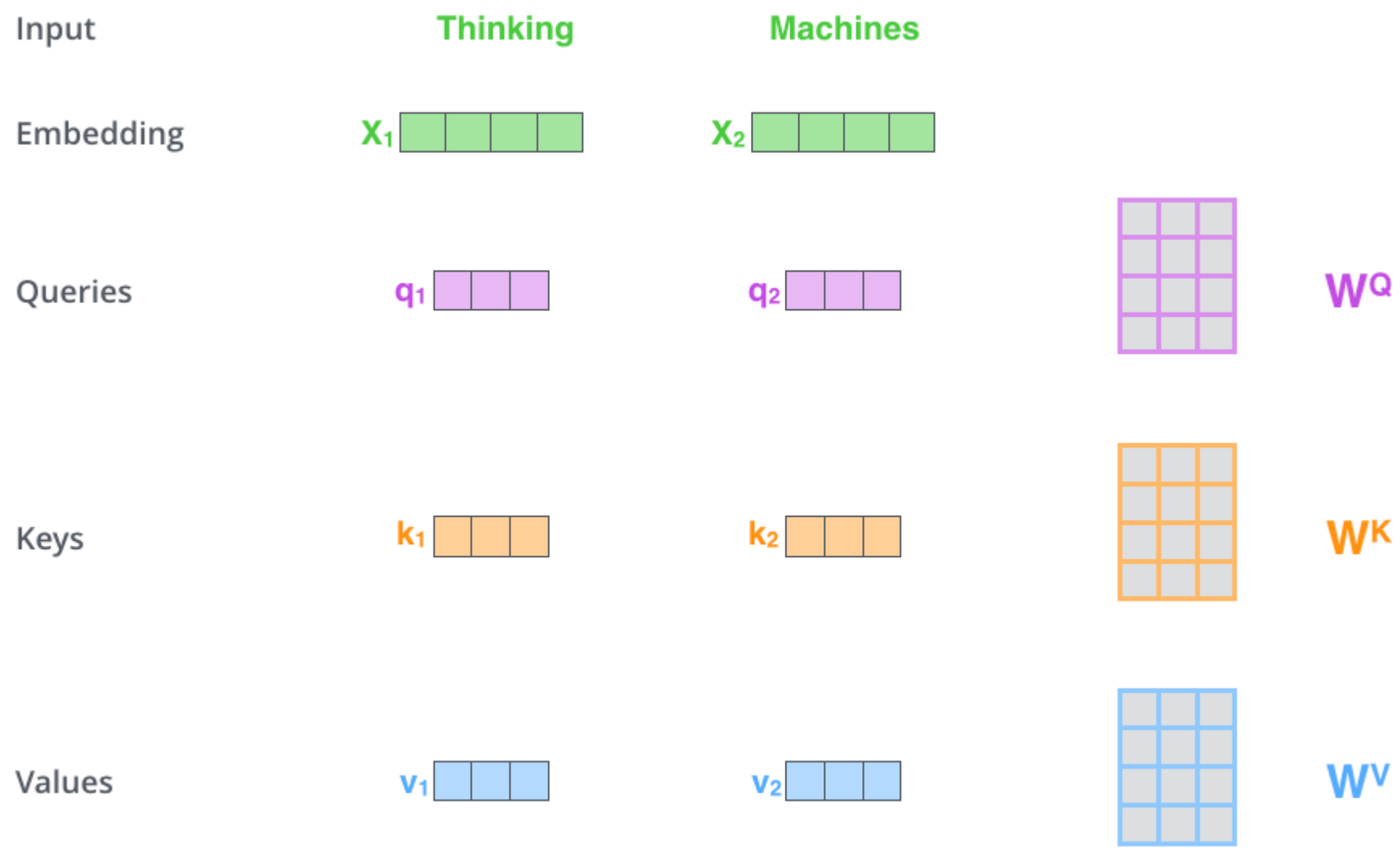
You can *toggle* a head by a **click** on the heatmap columns

Tokens on the *left* attend to tokens on the *right*.



<https://huggingface.co/exbert/>

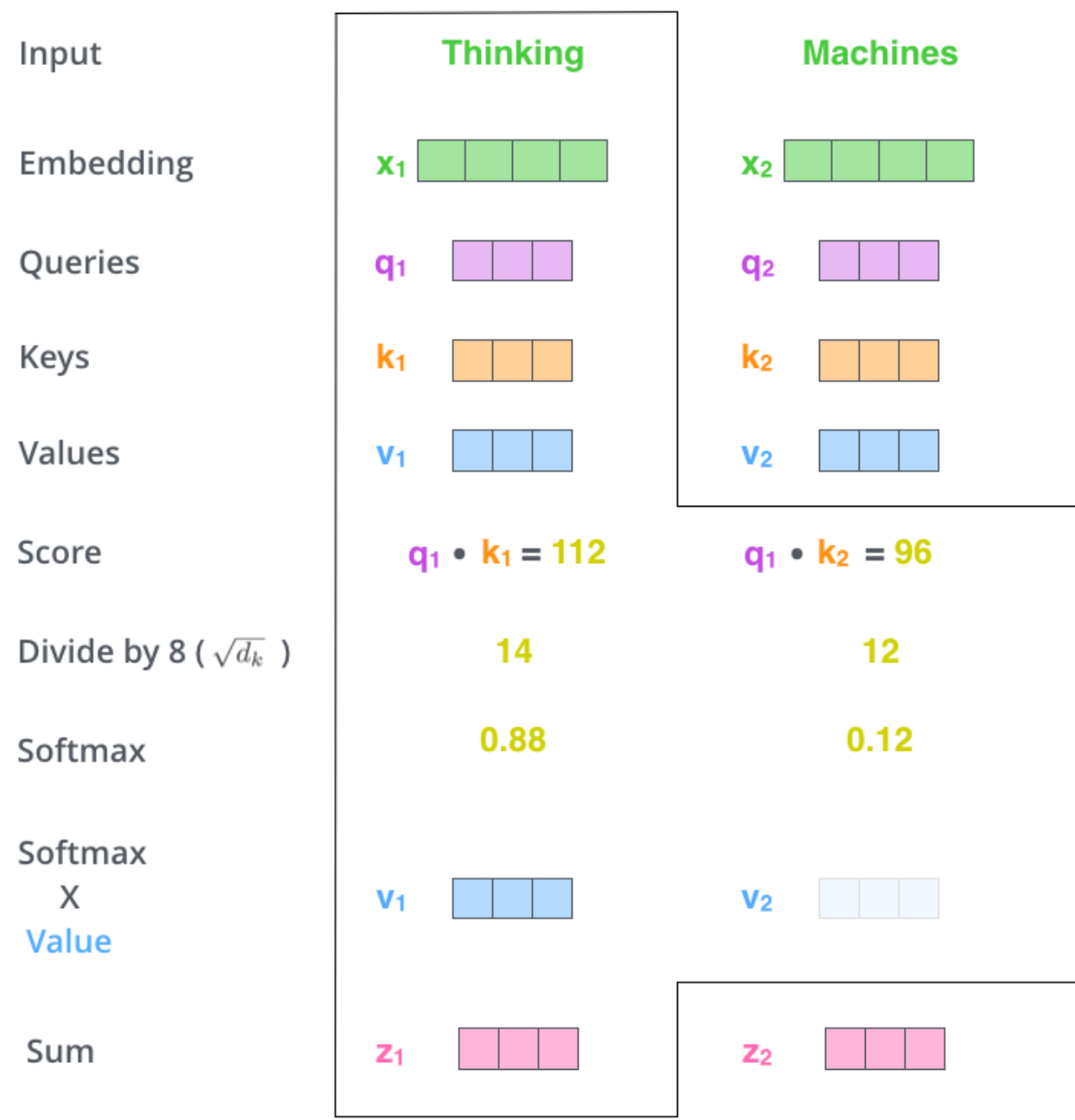
Реализация «Self-Attention»



три 64-мерных вектора: Query, Key, Value – получаются из представлений (embeddings) умножением на матрицу (для каждого QKV – своя) – это обучаемый параметр



Реализация «Self-Attention»



Реализация «Self-Attention»: простая матричная

$X \times W^Q = Q$

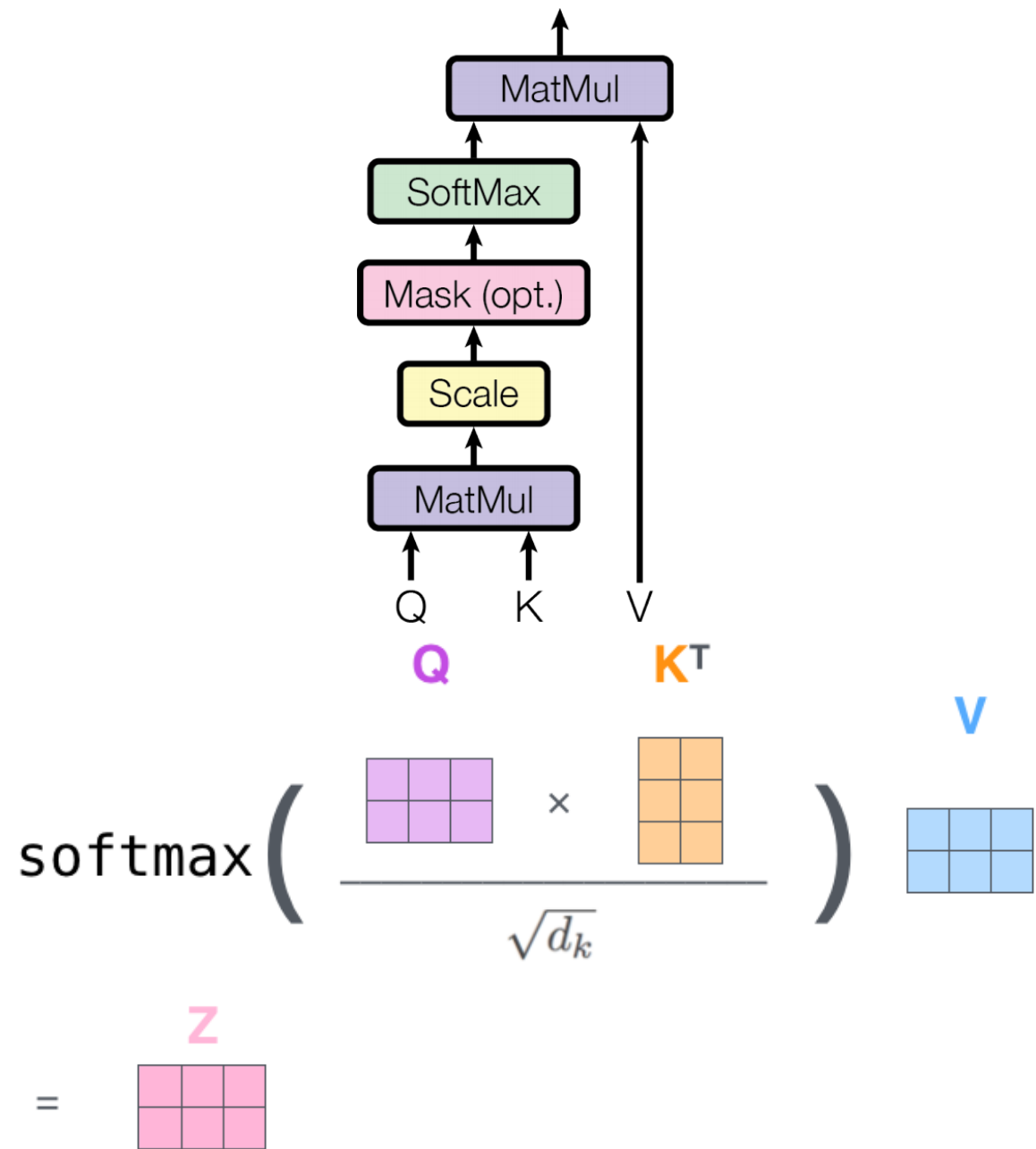


$X \times W^K = K$

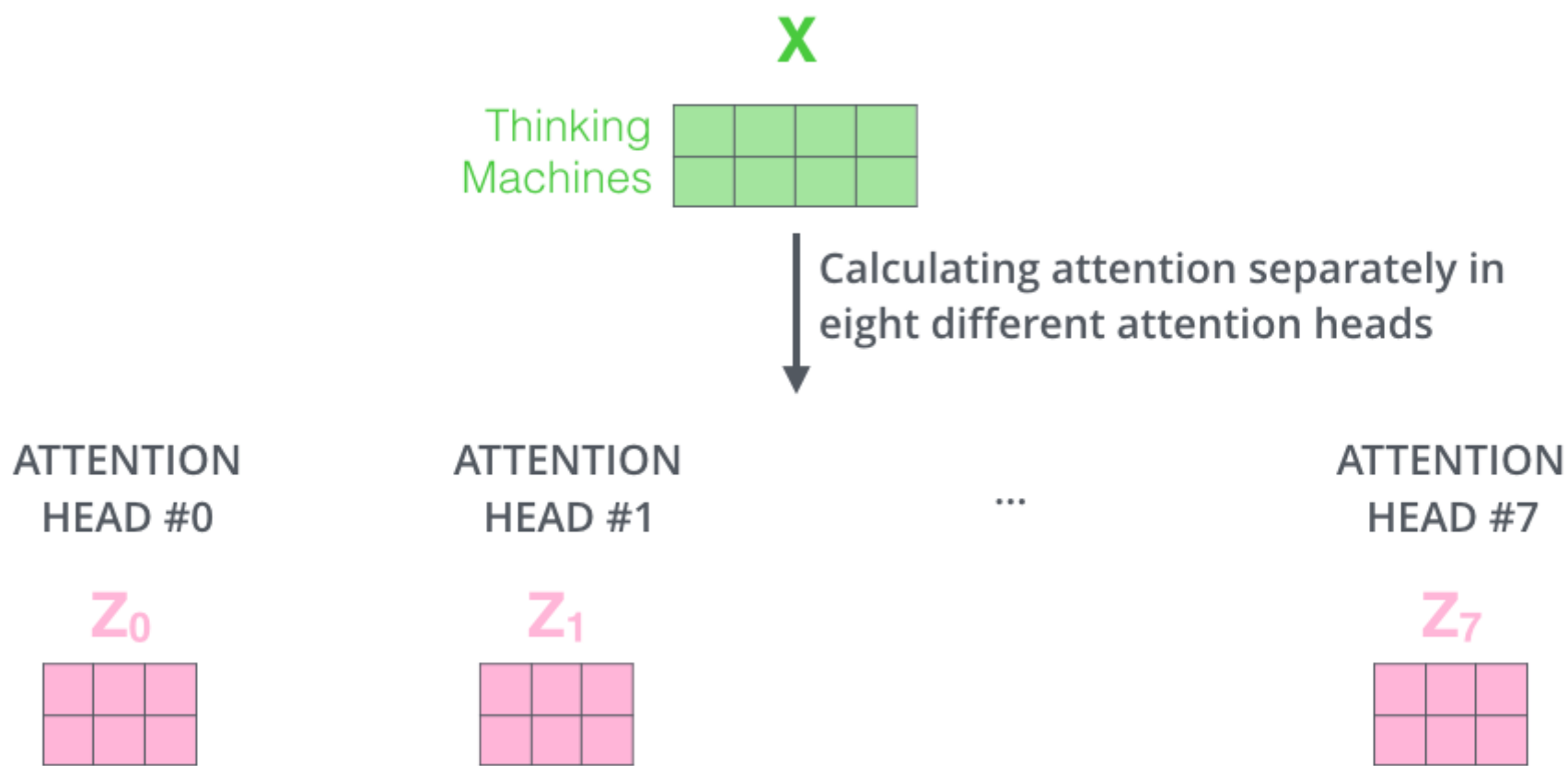


$X \times W^V = V$





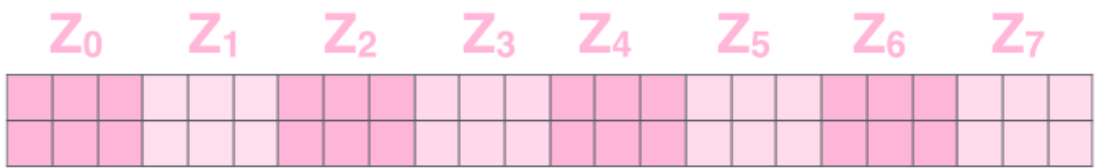
Multi-Headed Attention (несколько головок)



**каждая головка обозревает свой аспект внимания (что это, что делает и т.п.)**  
**как теперь превратить результаты разных головок в вектор нужной размерности?**

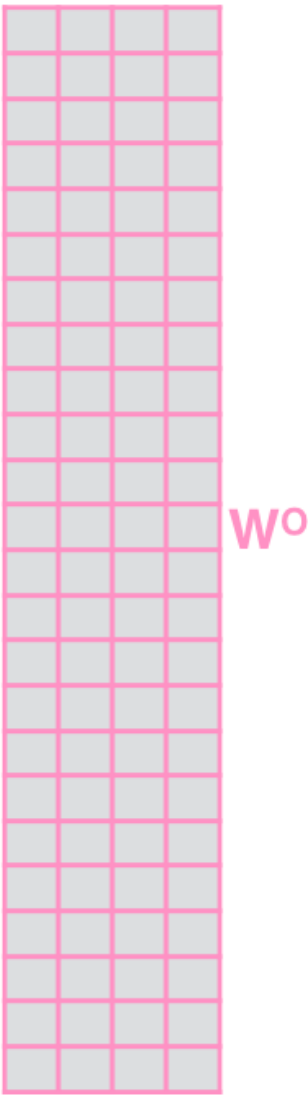
Multi-Headed Attention (несколько головок)

1) Concatenate all the attention heads

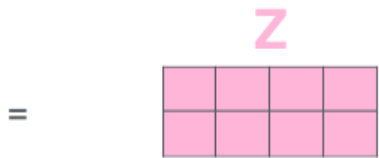


2) Multiply with a weight matrix  $W^O$  that was trained jointly with the model

$\times$



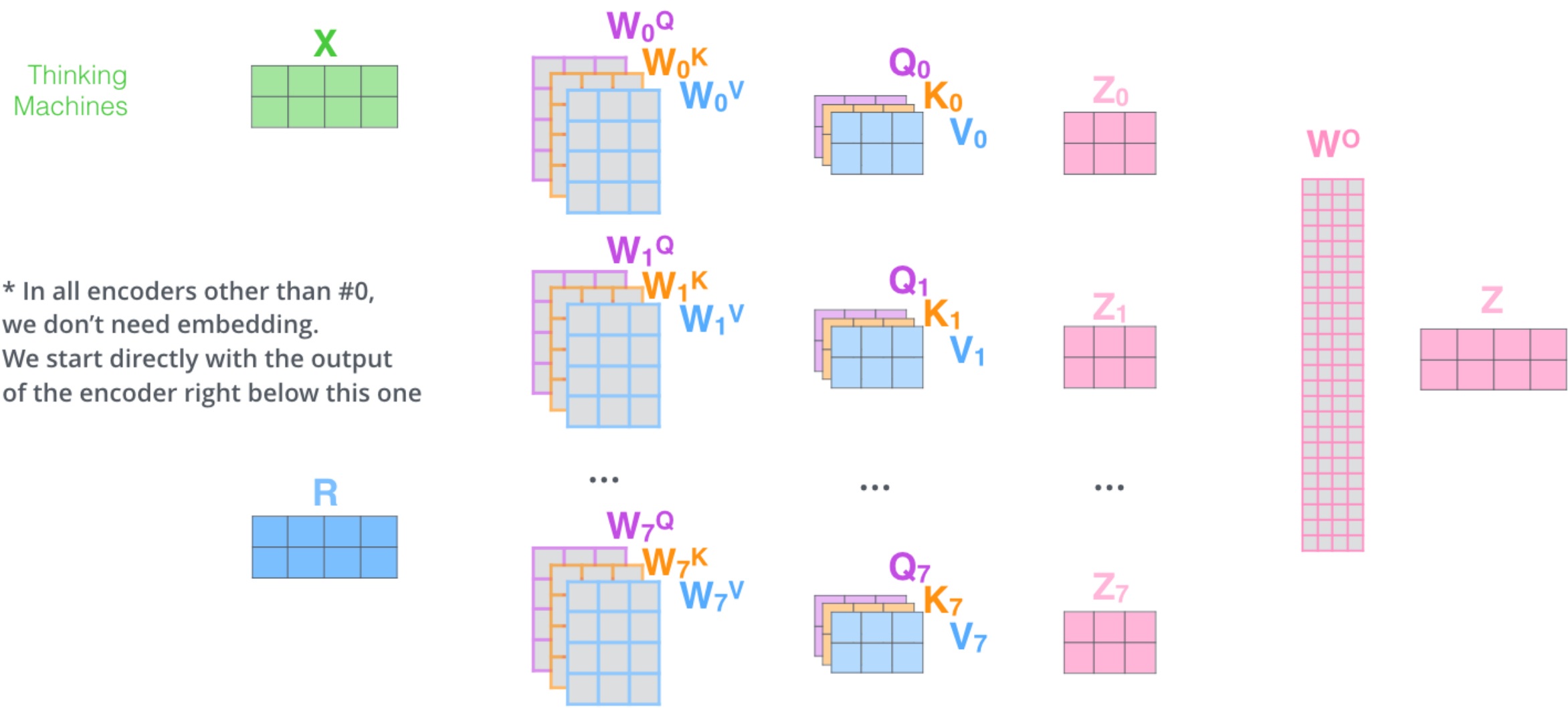
3) The result would be the  $Z$  matrix that captures information from all the attention heads. We can send this forward to the FFNN



конкатенация и умножение на ещё одну матрицу весов

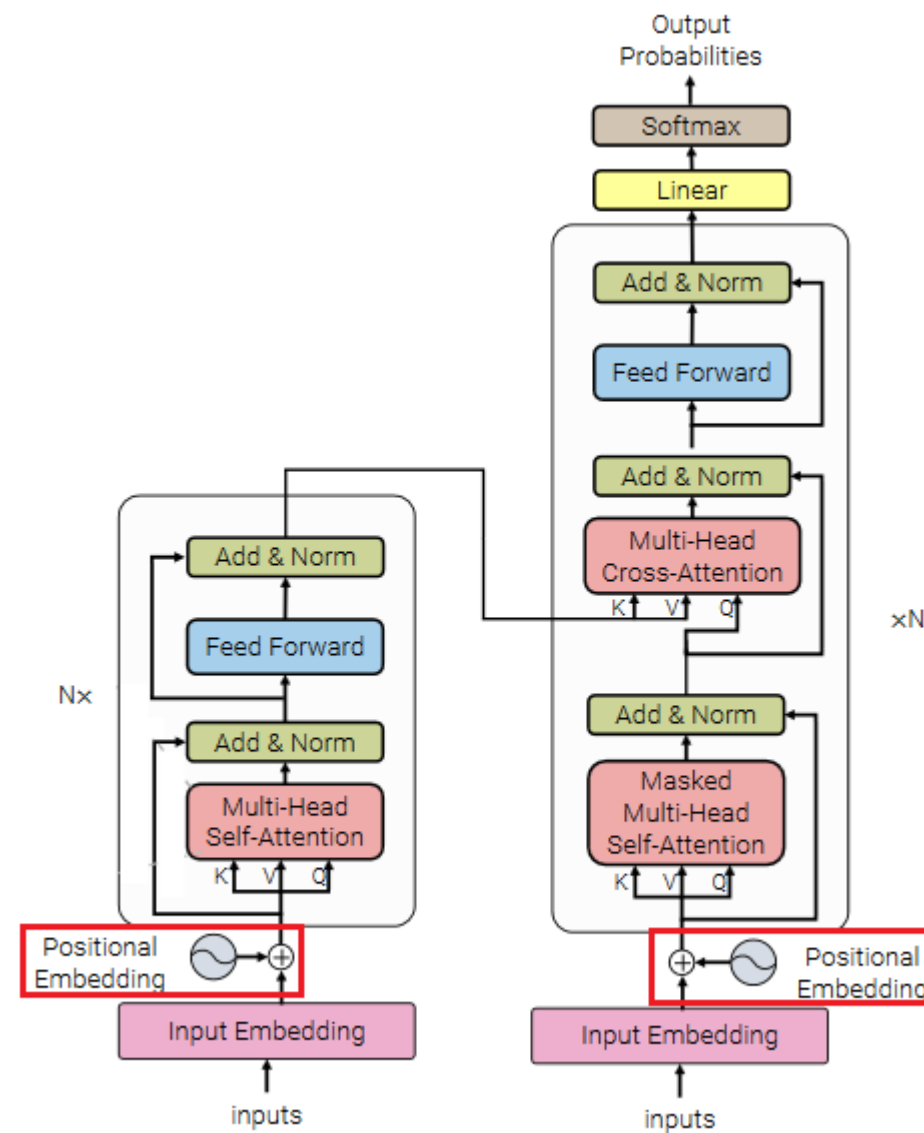
Multi-Headed Attention (несколько головок) – саммари

- 1) This is our input sentence\*
- 2) We embed each word\*
- 3) Split into 8 heads. We multiply  $X$  or  $R$  with weight matrices
- 4) Calculate attention using the resulting  $Q/K/V$  matrices
- 5) Concatenate the resulting  $Z$  matrices, then multiply with weight matrix  $W^O$  to produce the output of the layer



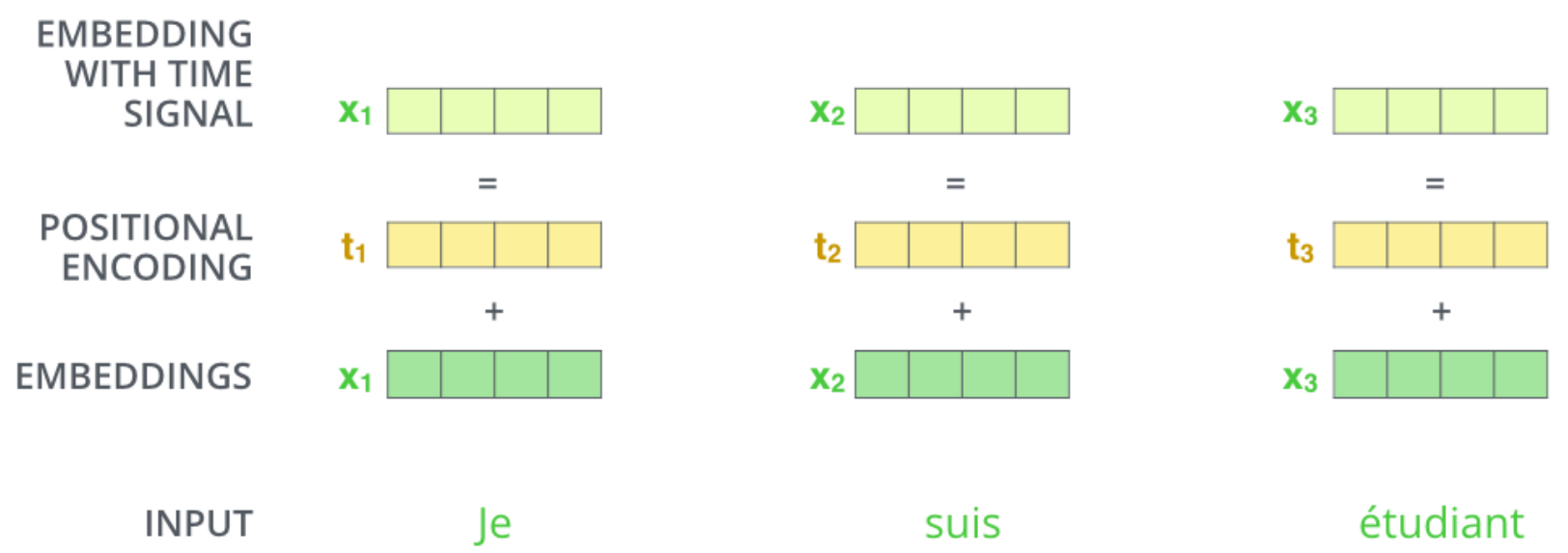
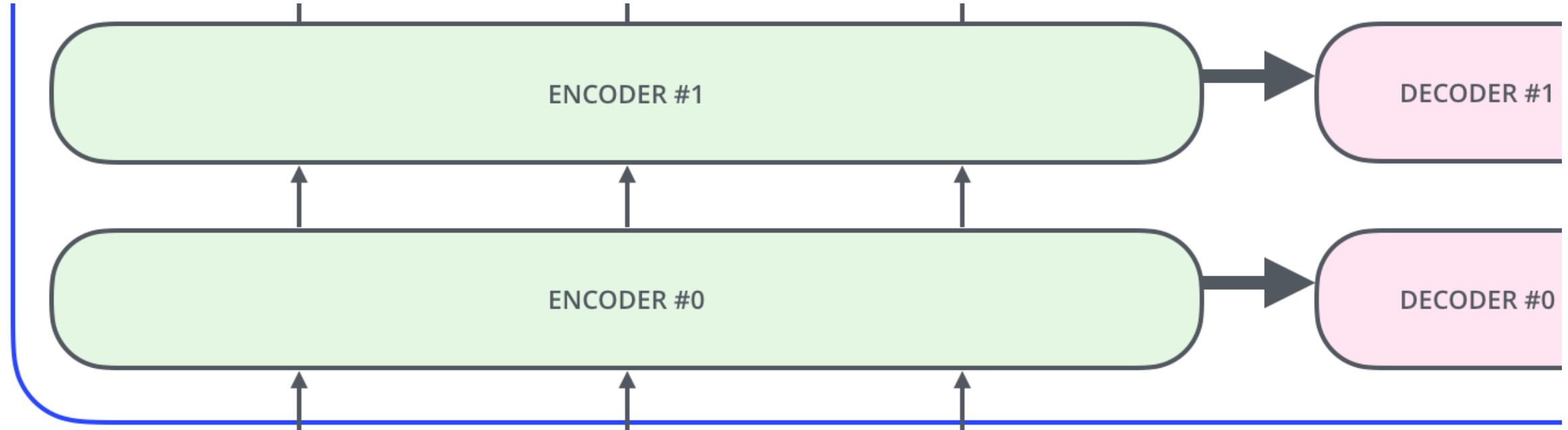
\* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one

## Кодирование позиции слова: Positional Encoding



**Проблема: трансформер пока инвариантен к перестановке слов**

Кодирование позиции слова: Positional Encoding

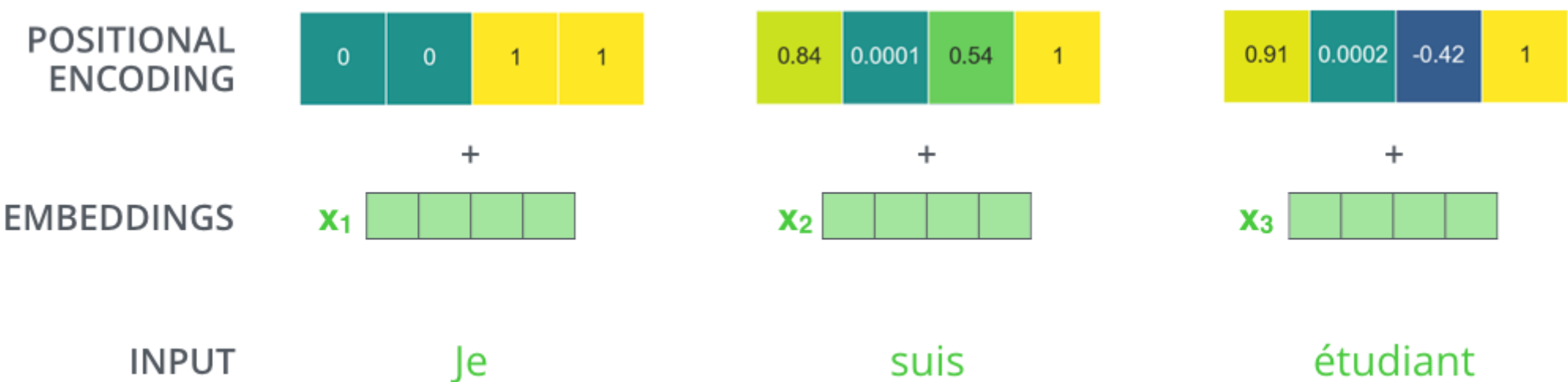


Кодирование позиции слова: Positional Encoding

Но надо, чтобы метод не зависел от длины входного предложения

номеру позиции  $t$  соответствует  $d$ -мерный вектор, позиции которого

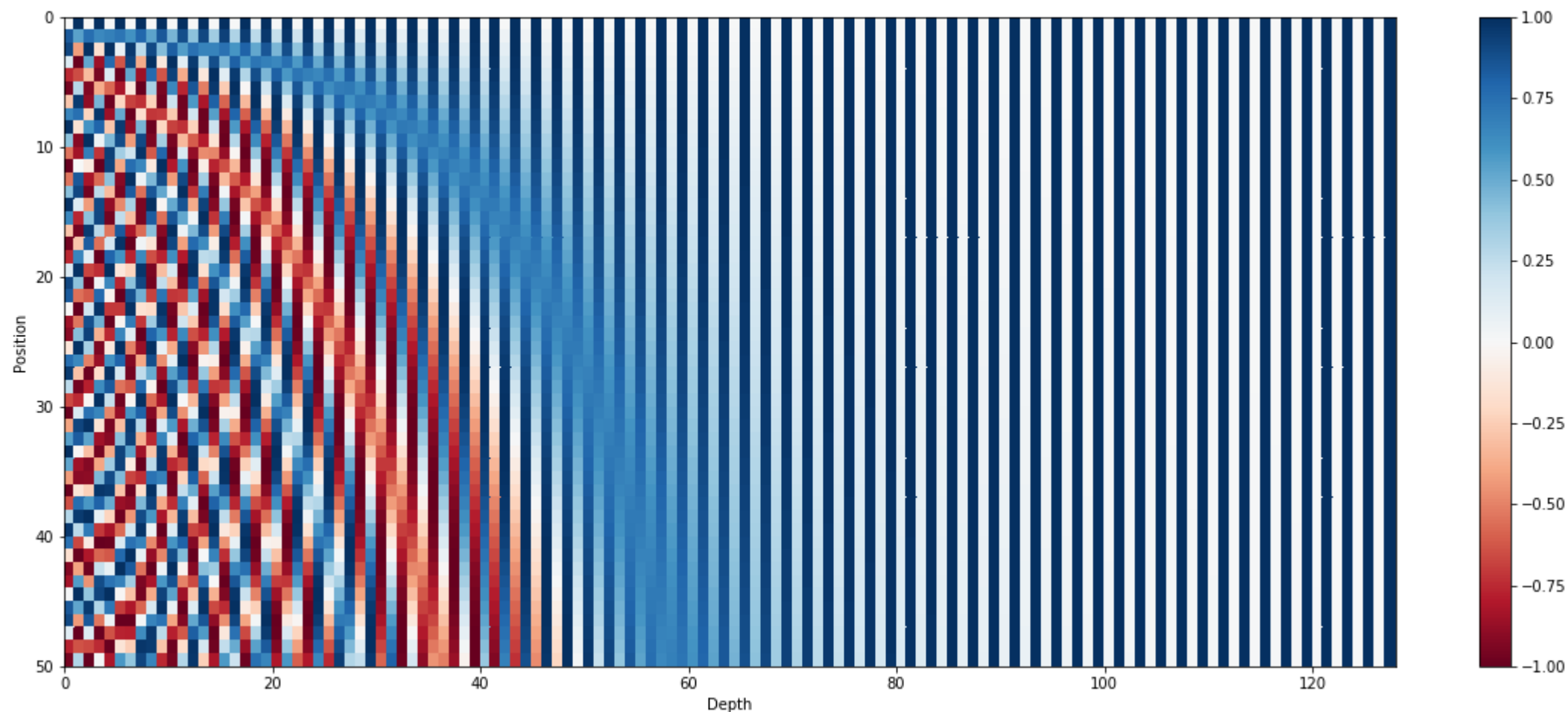
$$(2i, 2i + 1) \rightarrow \left( \sin\left(\frac{2t}{10000^{2i/d}}\right), \cos\left(\frac{2t}{10000^{2i/d}}\right) \right)$$



картинка для размерности представления = 4



## Кодирование позиции слова: Positional Encoding



**128-мерное кодирование 50 позиций**

[https://kazemnejad.com/blog/transformer\\_architecture\\_positional\\_encoding/](https://kazemnejad.com/blog/transformer_architecture_positional_encoding/)

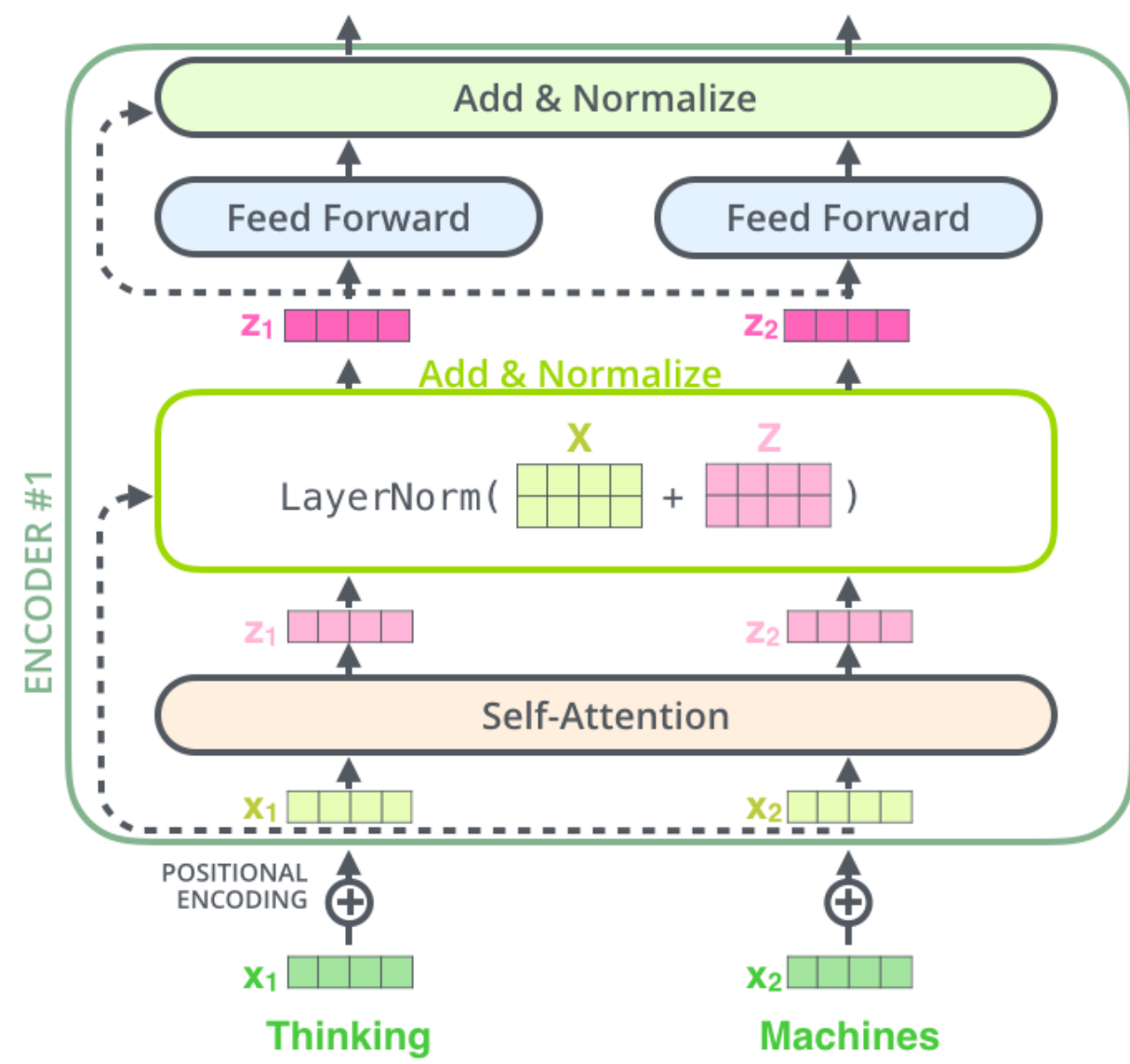
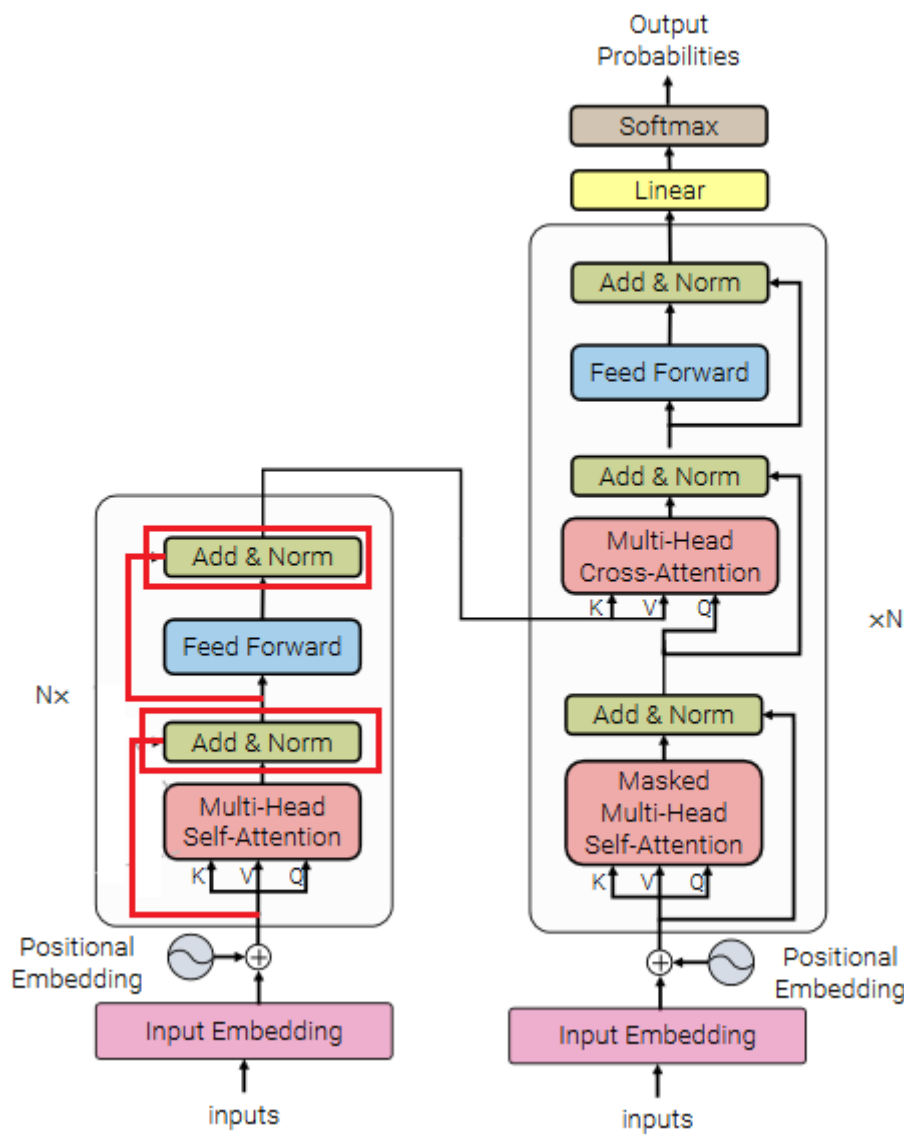
## Кодирование позиции слова: Positional Encoding

```
class PositionalEncoding(nn.Module):
    "Implement the PE function."
    def __init__(self, d_model, dropout, max_len=5000):
        super(PositionalEncoding, self).__init__()
        self.dropout = nn.Dropout(p=dropout)

        # Compute the positional encodings once in log space.
        pe = torch.zeros(max_len, d_model)
        position = torch.arange(0, max_len).unsqueeze(1)
        div_term = torch.exp(torch.arange(0, d_model, 2) *
                              - (math.log(10000.0) / d_model))
        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)
        pe = pe.unsqueeze(0)
        self.register_buffer('pe', pe) # не считается параметром модели

    def forward(self, x):
        x = x + Variable(self.pe[:, :x.size(1)], requires_grad=False)
        return self.dropout(x)
```

Transformer: residual connection and layer normalization



прокидывание связей – стандартный приём, нормировка тут особая...

## Transformer: residual connection and layer normalization

**Прокидывание связей – всё просто: размерности совпадают,  
можно сложить**

$$X_A = \text{LayerNorm}(\text{MultiheadSelfAttention}(X)) + X$$

$$X_B = \text{LayerNorm}(\text{PositionFFN}(X_A)) + X_A$$

```
class SublayerConnection(nn.Module):  
    """  
    A residual connection + layer norm. For code simplicity the norm is first  
    """  
    def __init__(self, size, dropout):  
        super(SublayerConnection, self).__init__()  
        self.norm = LayerNorm(size)  
        self.dropout = nn.Dropout(dropout)  
  
    def forward(self, x, sublayer):  
        "Apply residual connection to any sublayer with the same size."  
        return x + self.dropout(sublayer(self.norm(x)))
```

## Transformer: Layer Norm

$$x = (x_1, \dots, x_d), \mu = \frac{1}{d} \sum_{i=1}^d x_i, \sigma^2 = \frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2$$

$$\text{LN}(x) = \gamma \frac{x - \mu}{\sigma} + \beta$$

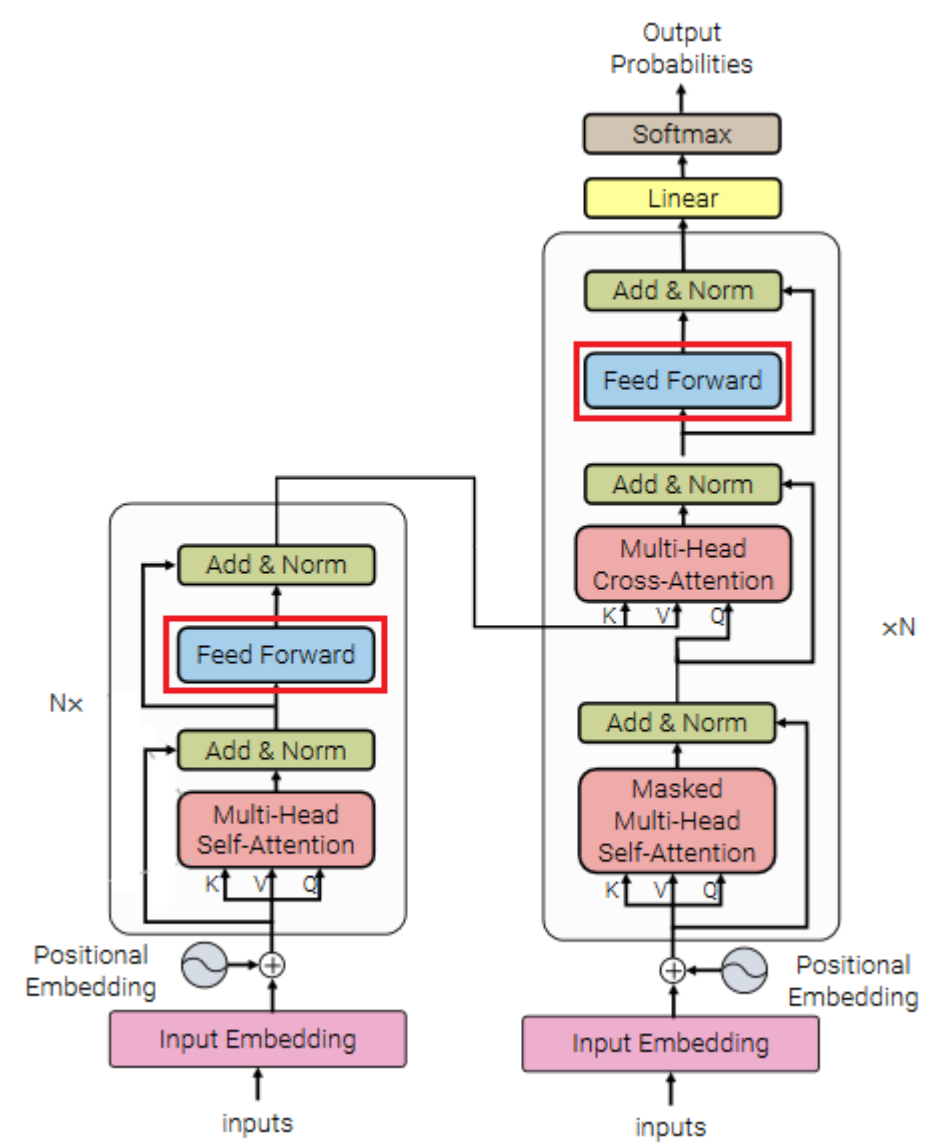
```
class LayerNorm(nn.Module):
    "Construct a layernorm module (See citation for details)."
```

```
def __init__(self, features, eps=1e-6):
    super(LayerNorm, self).__init__()
    self.a_2 = nn.Parameter(torch.ones(features))
    self.b_2 = nn.Parameter(torch.zeros(features))
    self.eps = eps

def forward(self, x):
    mean = x.mean(-1, keepdim=True)
    std = x.std(-1, keepdim=True)
    return self.a_2 * (x - mean) / (std + self.eps) + self.b_2
```

<http://nlp.seas.harvard.edu/2018/04/03/attention.html>

Transformer: FFNN



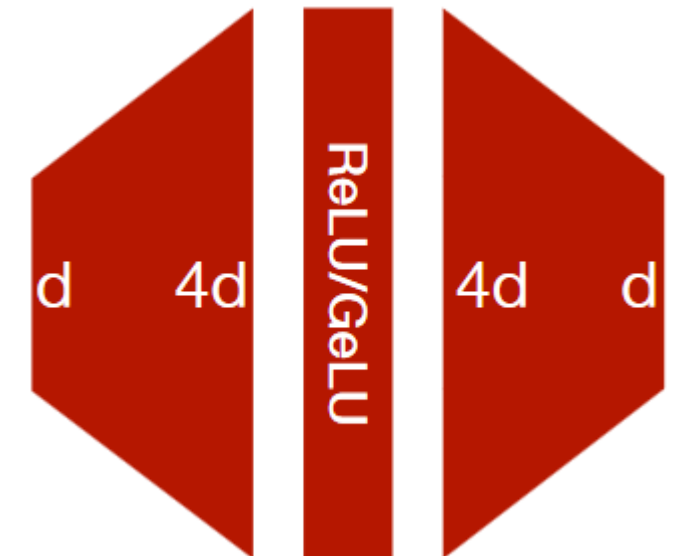
Полносвязный модуль – для преобразования представления токена

**Transformer: FFNN + inverted bottleneck**

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

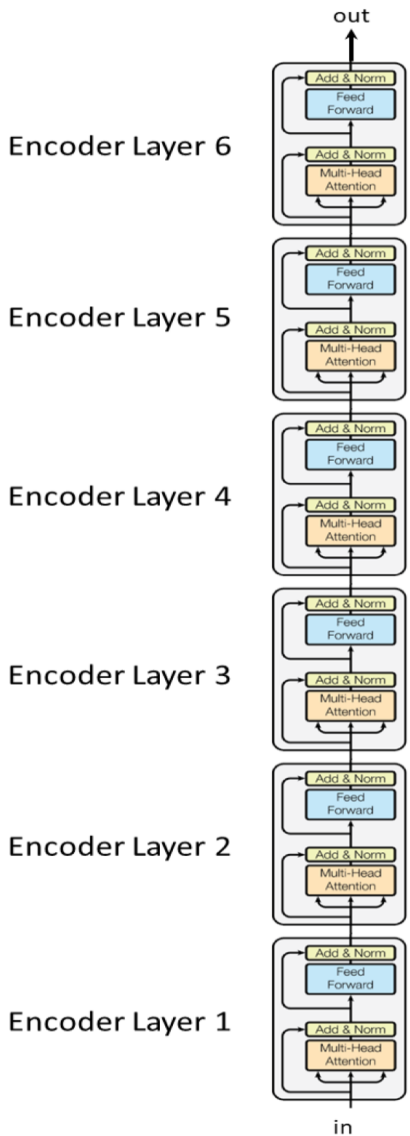
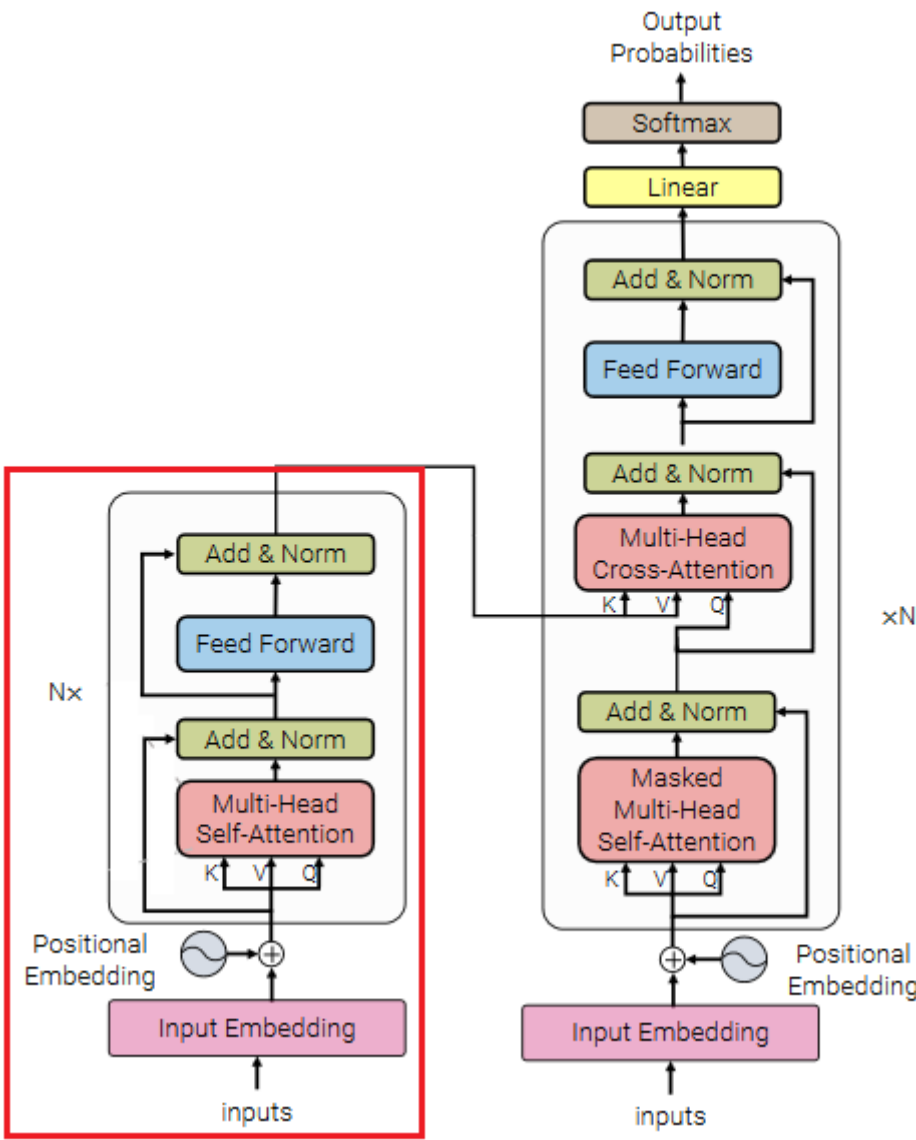
```
class PositionwiseFeedForward(nn.Module):
    "Implements FFN equation."
    def __init__(self, d_model, d_ff, dropout=0.1):
        super(PositionwiseFeedForward, self).__init__()
        self.w_1 = nn.Linear(d_model, d_ff)
        self.w_2 = nn.Linear(d_ff, d_model)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        return
        self.w_2(self.dropout(F.relu(self.w_1(x))))
```



**Parameters sharing – на всех токенах обрабатывает одна сеть**

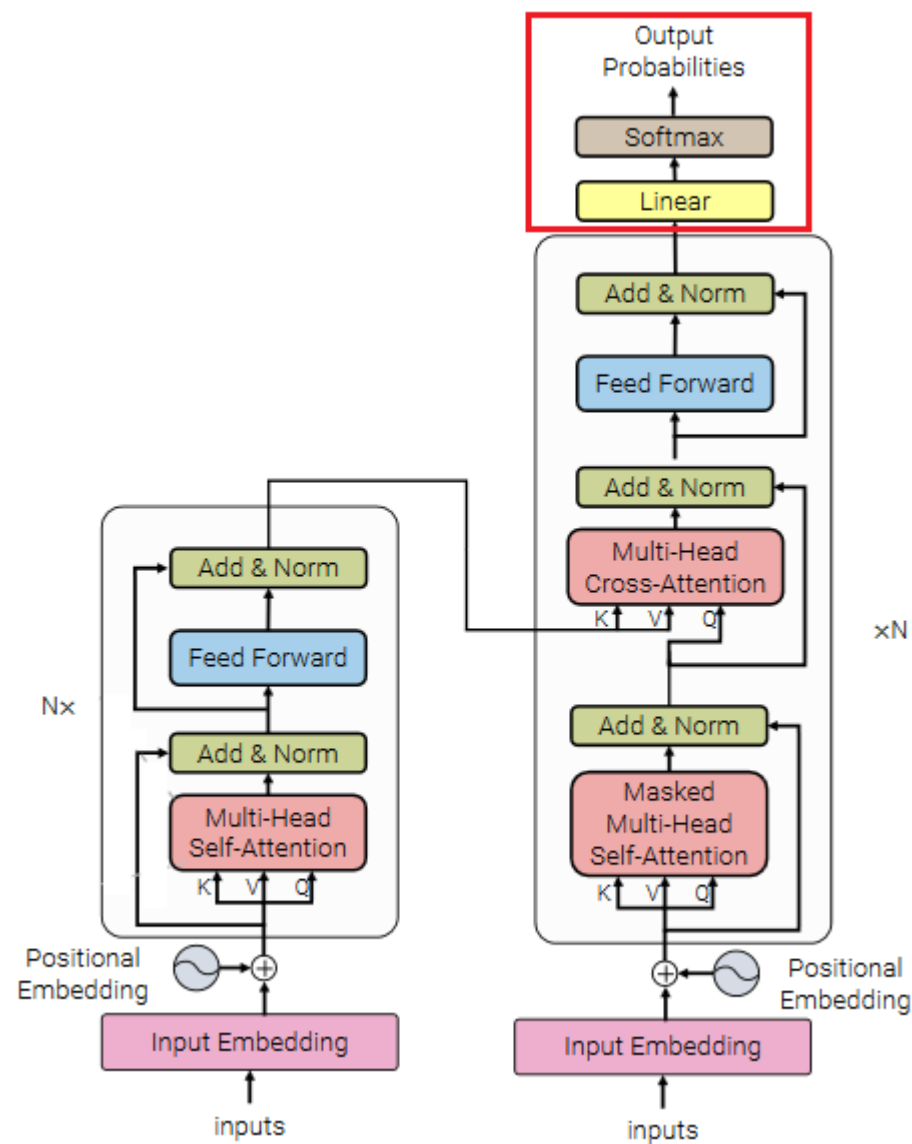
Transformer: Encoder



полностью знаем устройство кодировщика: модули преобразуют представления токенов



Transformer: Decoder



Что выдаёт декодировщик? Токены!

Transformer: Decoder Output layer

как на выходе получить одно слово:

Which word in our vocabulary  
is associated with this index?

Get the index of the cell  
with the highest value  
(argmax)

am

5

log\_probs



Softmax

logits



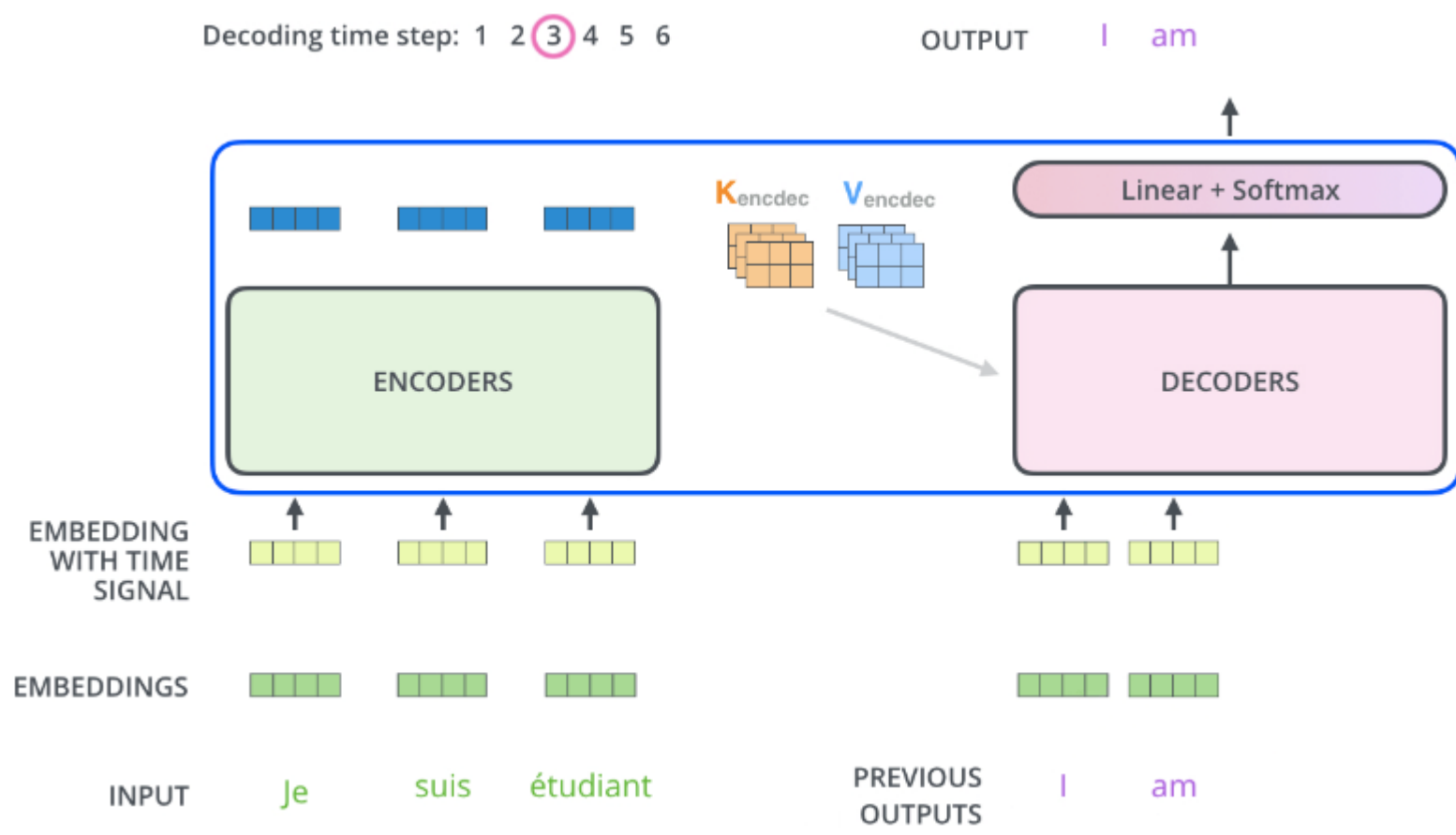
Linear

Decoder stack output



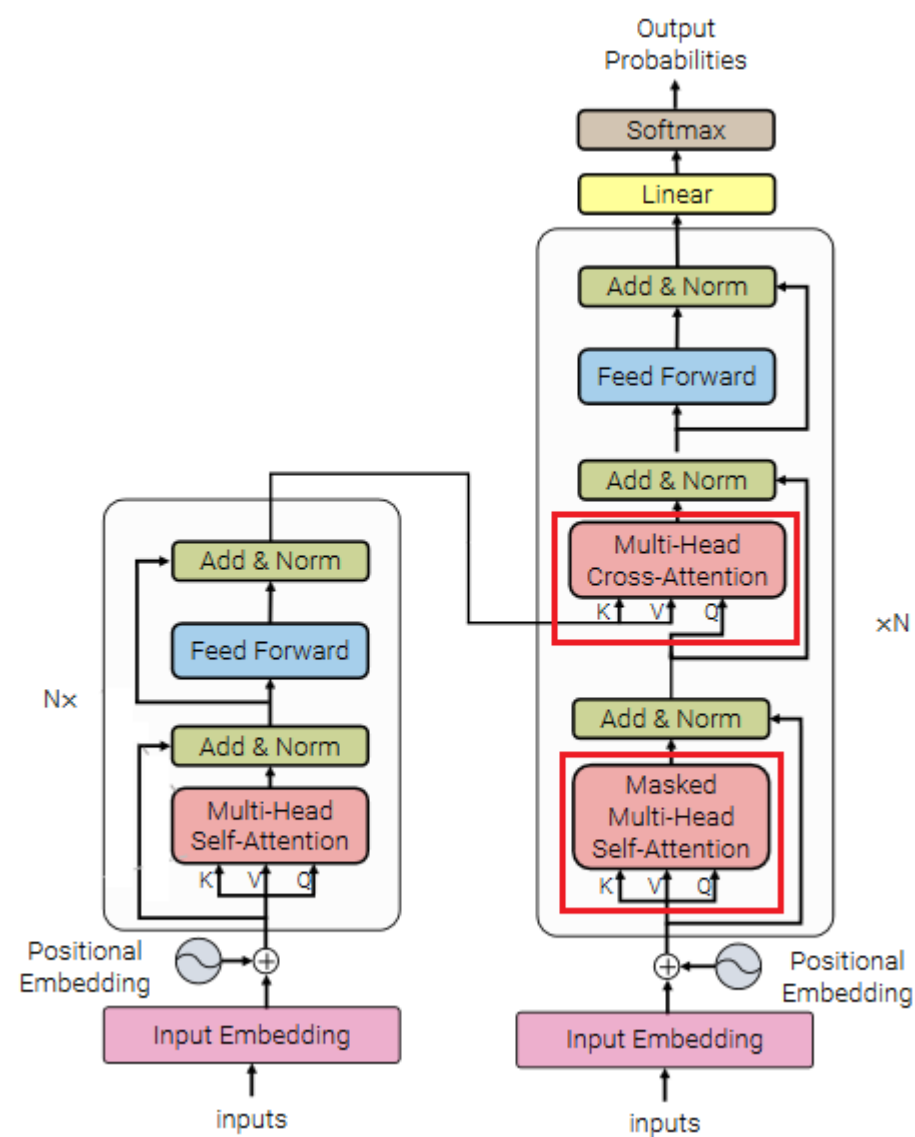
# Transformer: Decoder

**Тонкость: не знаем длину ответа**  
⇒ **будем его получать последовательно**



**На инференсе рекуррентная генерация, обучаться будем нерекуррентно!**

Transformer: виды внимания

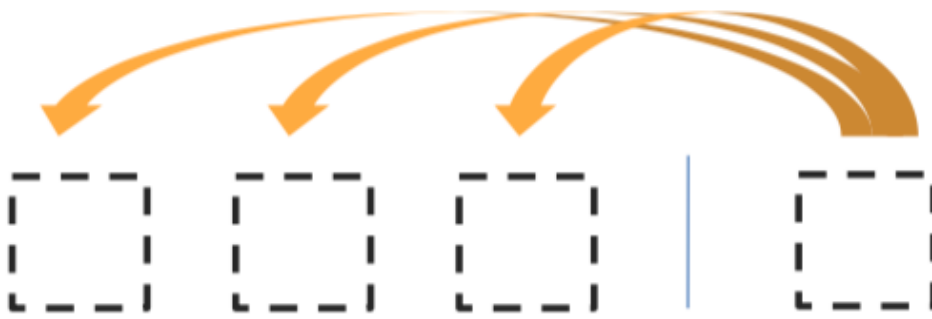


К обычному MHSA (в кодировщике) добавляются MMHSA и MHCA

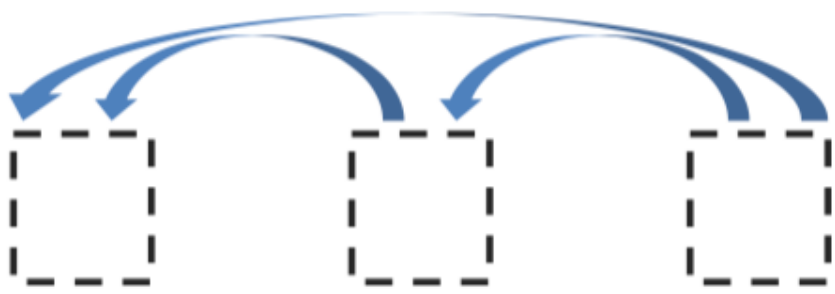
# Transformer: виды внимания



Encoder Self-Attention



Encoder-Decoder Attention



MaskedDecoder Self-Attention

**в кодировщике смотрим на всё предложение, чтобы понять смысл всех токенов и получить идеальные представления**

**в декодировщике смотрим на кодировщик (чтобы перевести предложение, надо смотреть на оригинал)**

**в декодировщике при генерации нельзя знать будущее (что сгенерировали после этого токена)**

## Transformer: маскирование

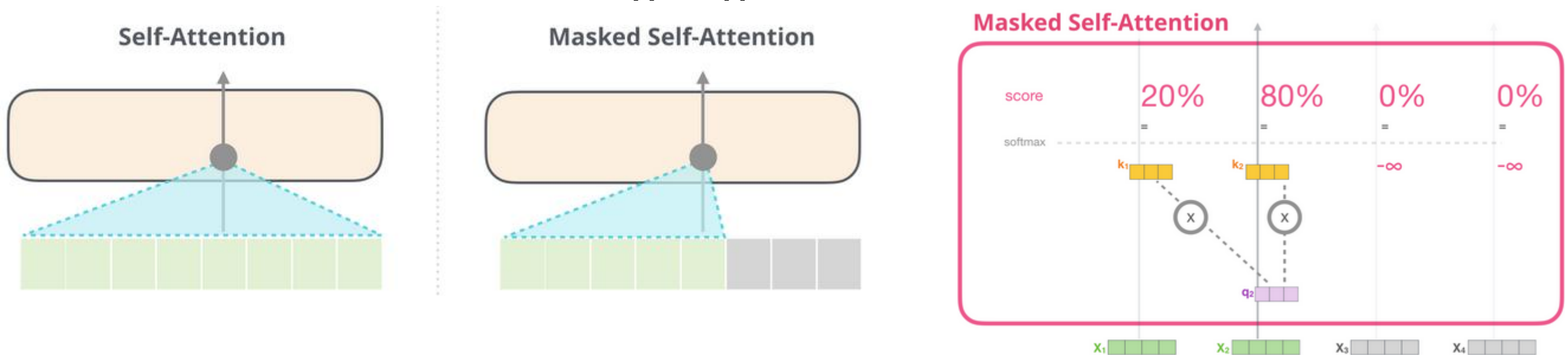
**padding mask** (in all the scaled dot-product attention)

~ добавляем 0 в конец коротких предложений

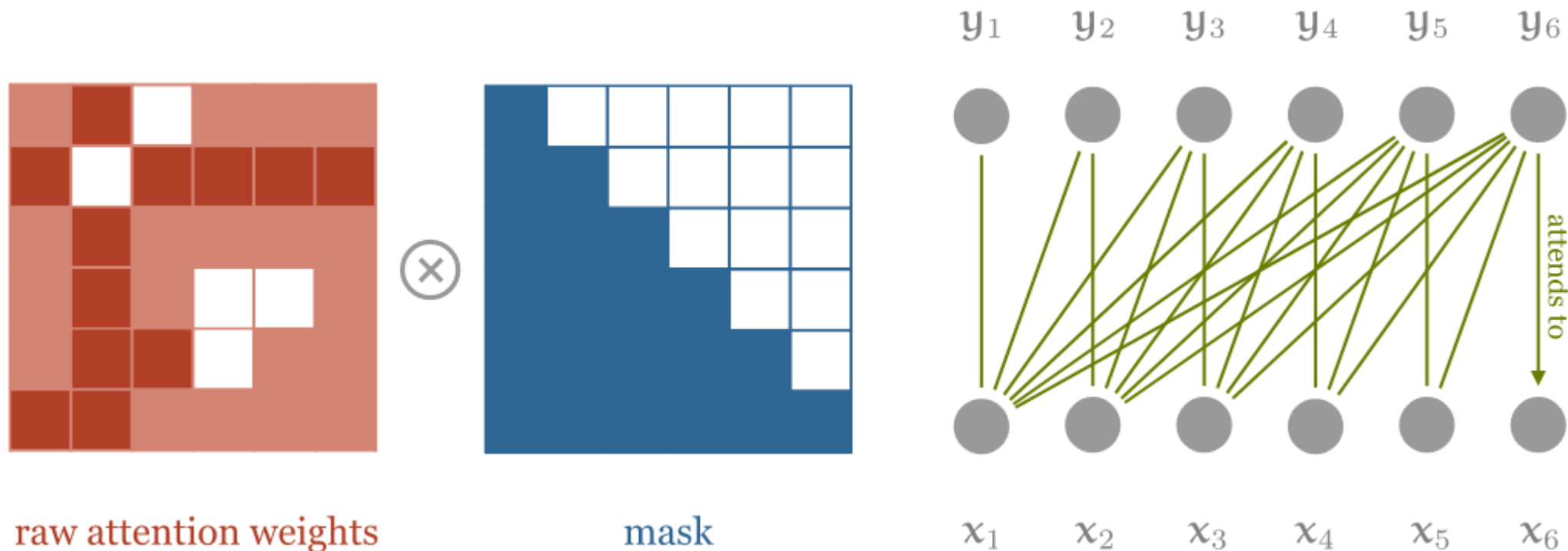
**sequence mask** (in the decoder's self-attention)

чтобы не видел информацию из будущего

**ВЫВОДИТ ОДНО СЛОВО ЗА ТАКТ**



## Transformer: маскирование

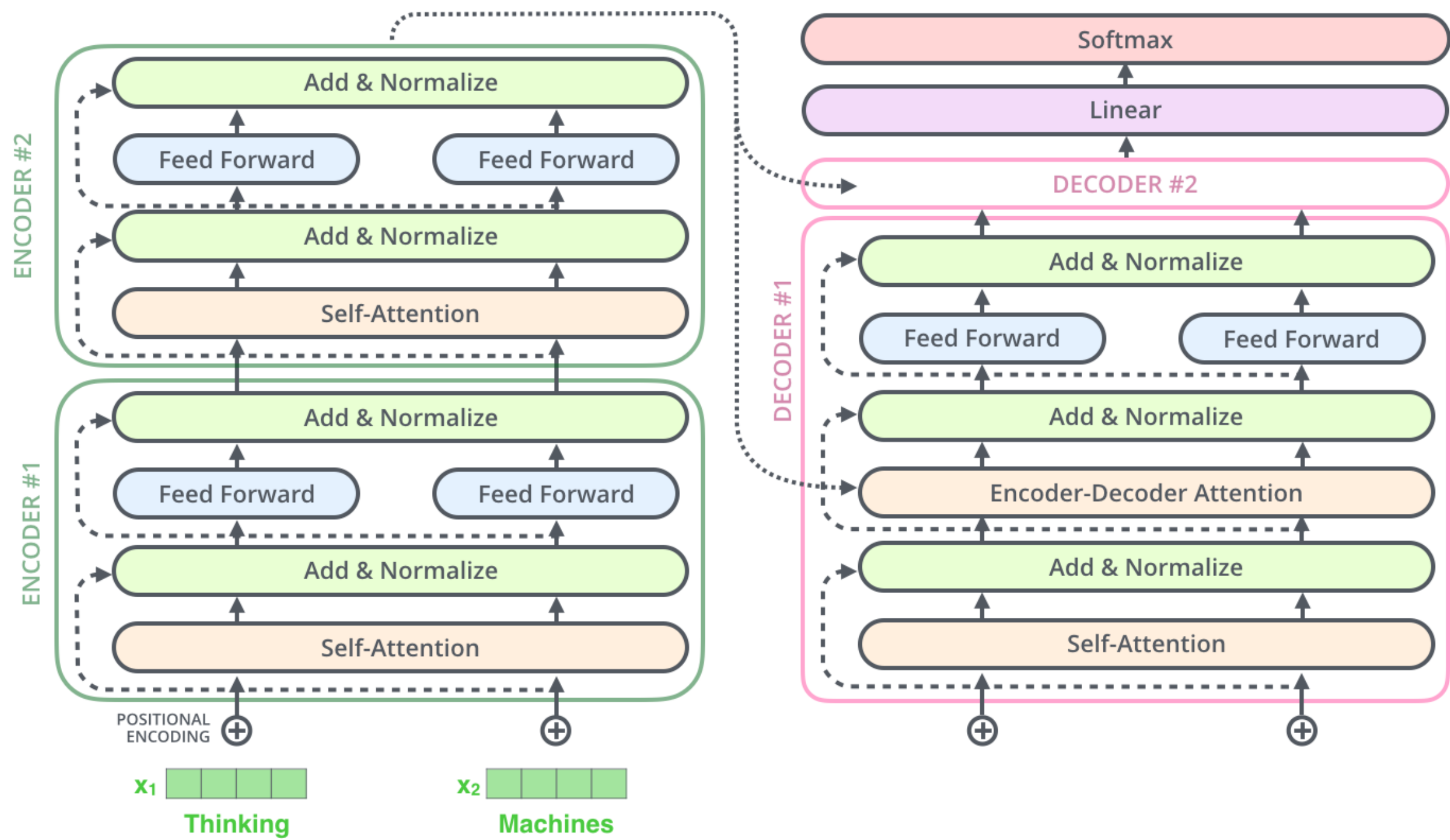


raw attention weights

mask

```
dot = torch.bmm(queries, keys.transpose(1, 2))
indices = torch.triu_indices(t, t, offset=1)
dot[:, indices[0], indices[1]] = float('-inf')
dot = F.softmax(dot, dim=2)
```

<http://peterbloem.nl/blog/transformers>





## Transformer: главное

**трансформер – стекинг трансформер-блоков**  
послойное уточнение представлений токенов

**представления токенов не меняют размерность по слоям  $D$**   
не путать с внутренними размерностями  $d$  (запросов, ключей и значений)  
и исходными  $D_{\text{ONE}}$  (на вход трансформеру)

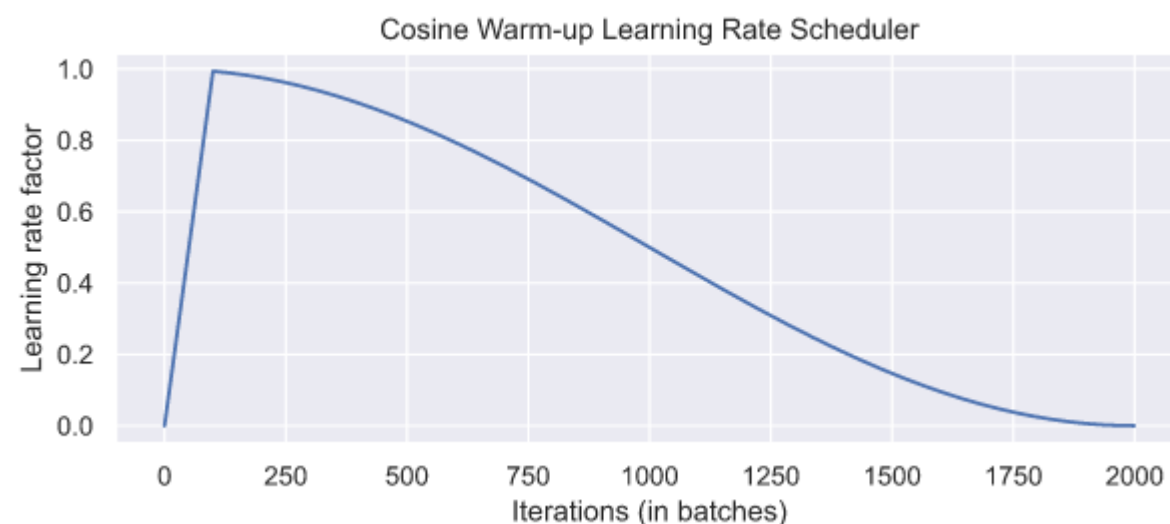
**self-attention можно рассмотреть как граф + пулинг по релевантности**  
**relevance-based pooling**

**нерекуррентная модель для последовательностей**

**глубокая модель с блоками внимания**

**изначально для NMT**  
**последний слой softmax + cross-entropy error**

## Обучение трансформера: Learning rate warm-up



**трансформеры обучают  
с такой программой изменения LR**

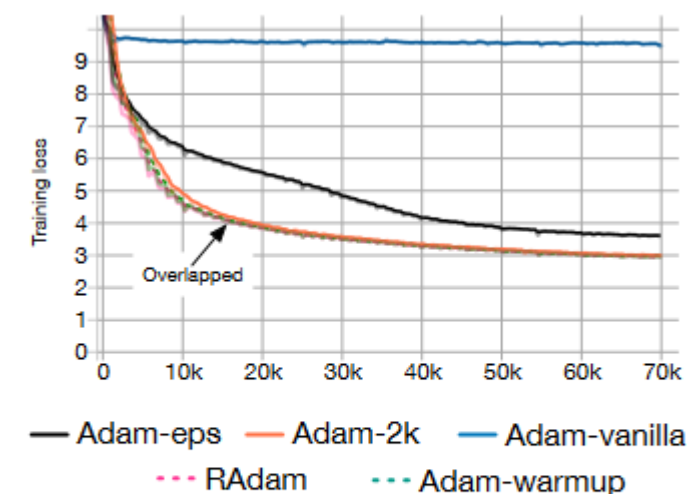


Figure 1: Training loss v.s. # of iterations of Transformers on the De-En IWSLT'14 dataset.

Рис. из <https://arxiv.org/pdf/1908.03265.pdf>

**В Adam используется bias correction factors –  
он увеличивает дисперсию на первых итерациях**

**Layer Normalization – может увеличивать норму градиента**

[https://huggingface.co/transformers/main\\_classes/optimizer\\_schedules.html?highlight=cosine#transformers.get\\_cosine\\_schedule\\_with\\_warmup](https://huggingface.co/transformers/main_classes/optimizer_schedules.html?highlight=cosine#transformers.get_cosine_schedule_with_warmup)

## Особенности обучения трансформера

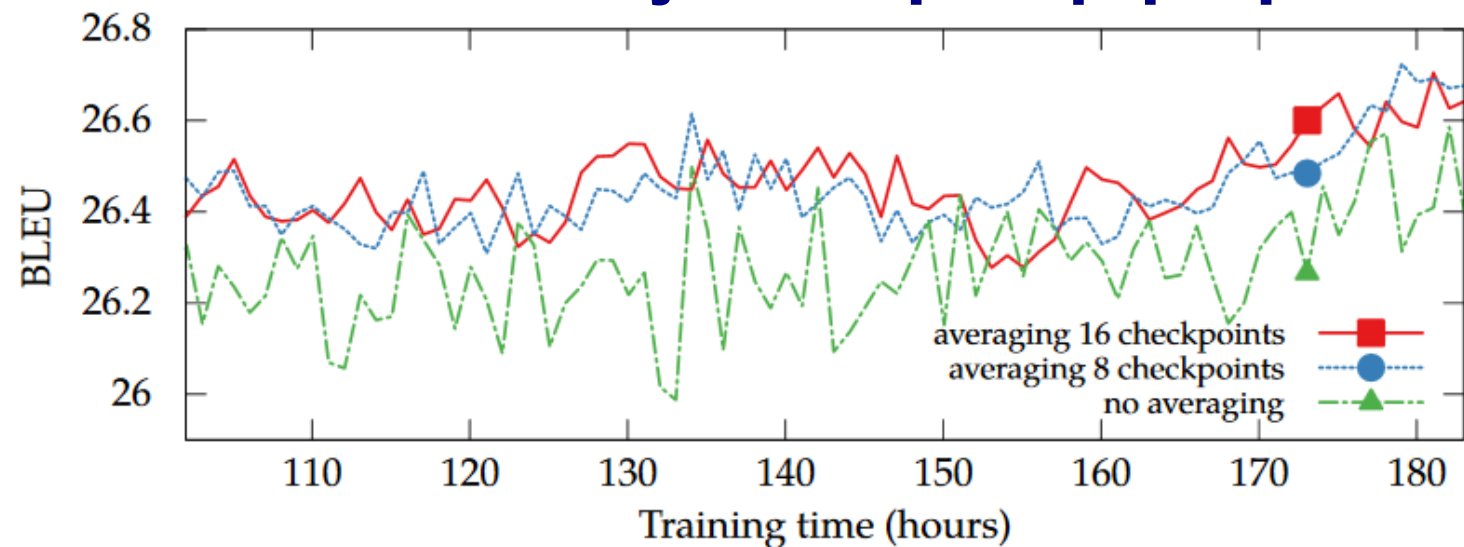


Figure 10: Effect of checkpoint averaging. All trained on 6 GPUs.

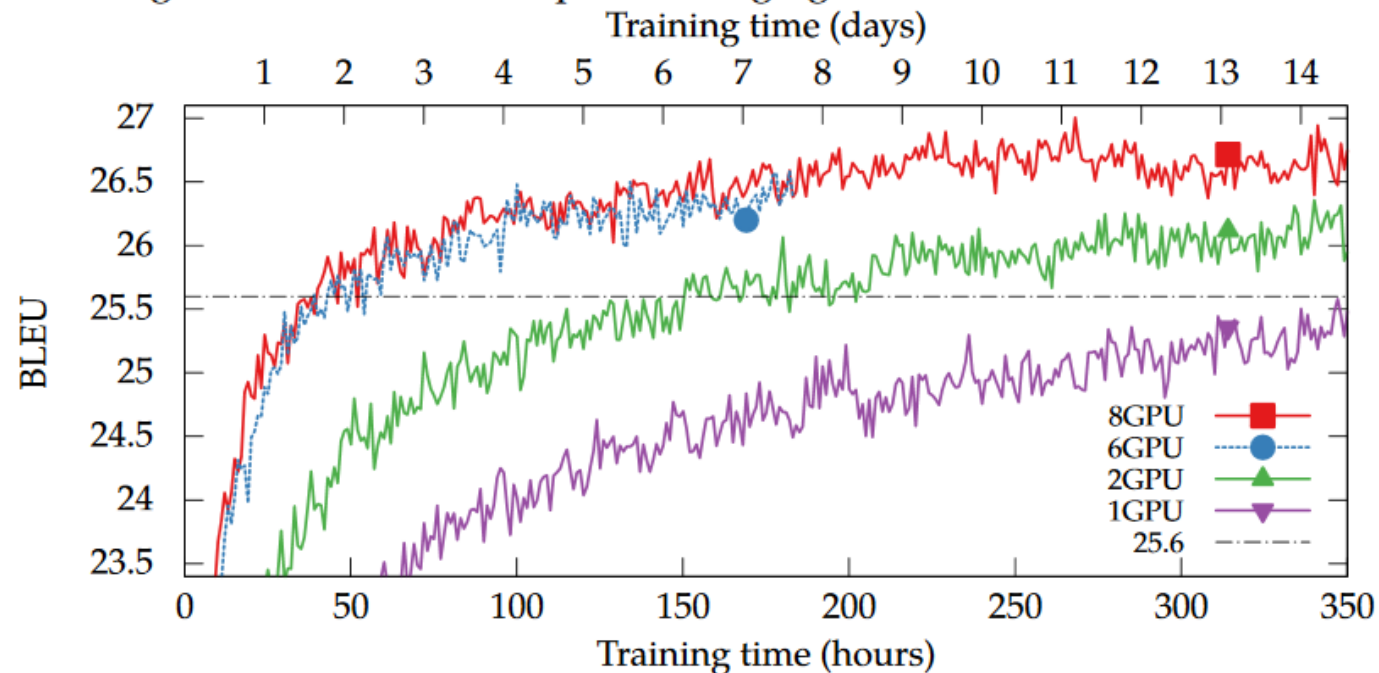


Figure 9: Effect of the number of GPUs. BLEU=25.6 is marked with a black line.

## Особенности обучения трансформера

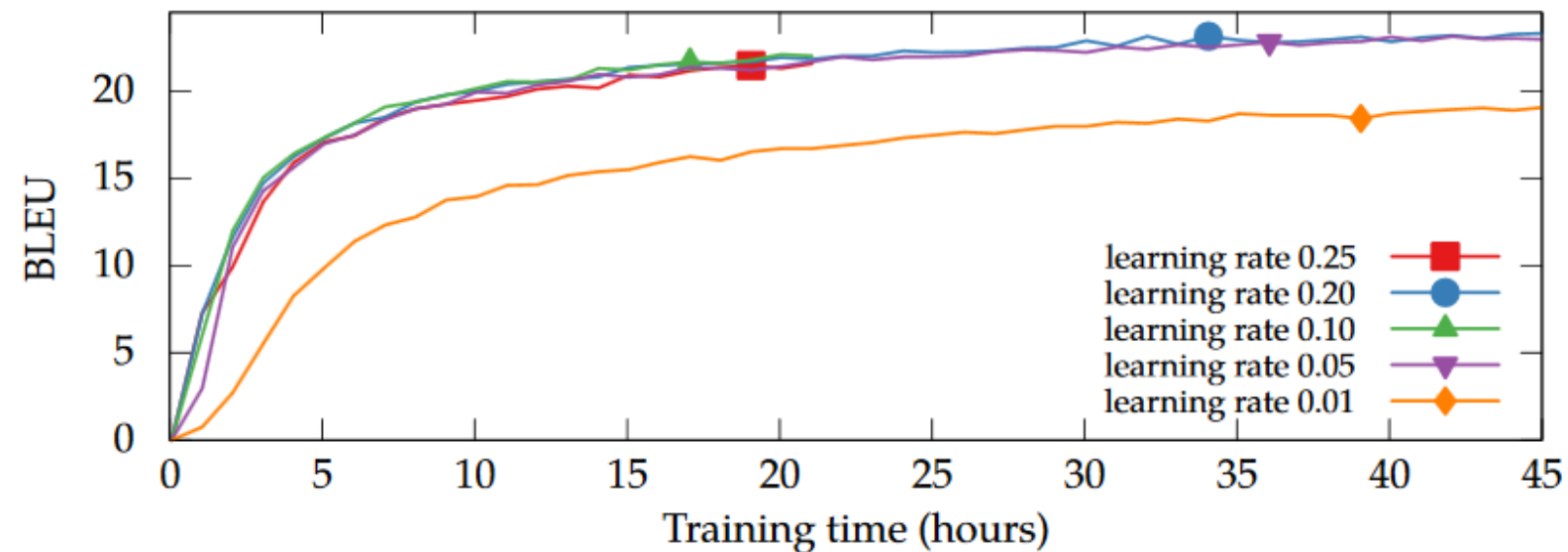


Figure 7: Effect of the learning rate on a single GPU. All trained on CzEng 1.0 with the default batch size (1500) and warmup steps (16k).

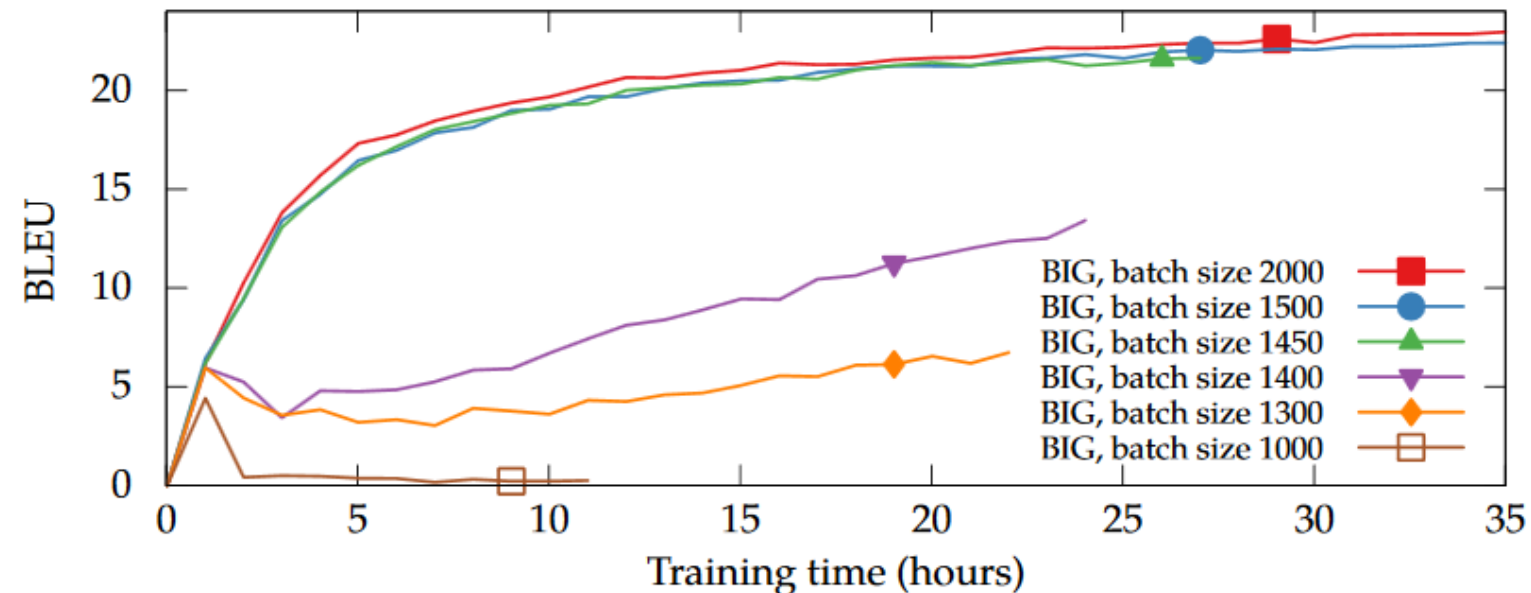


Figure 6: Effect of the batch size with the BIG model. All trained on a single GPU.

## Особенности обучения трансформера

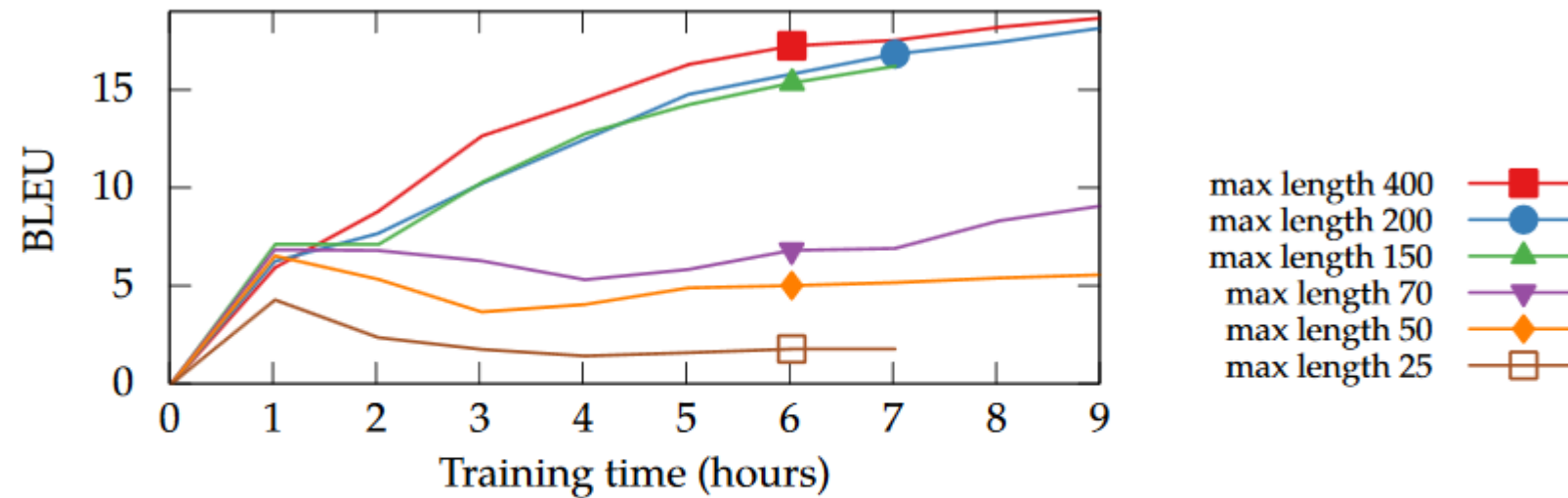


Figure 4: Effect of restricting the training data to various max\_length values. All trained on a single GPU with the BIG model and batch\_size=1500. An experiment without any max\_length is not shown, but it has the same curve as max\_length=400.

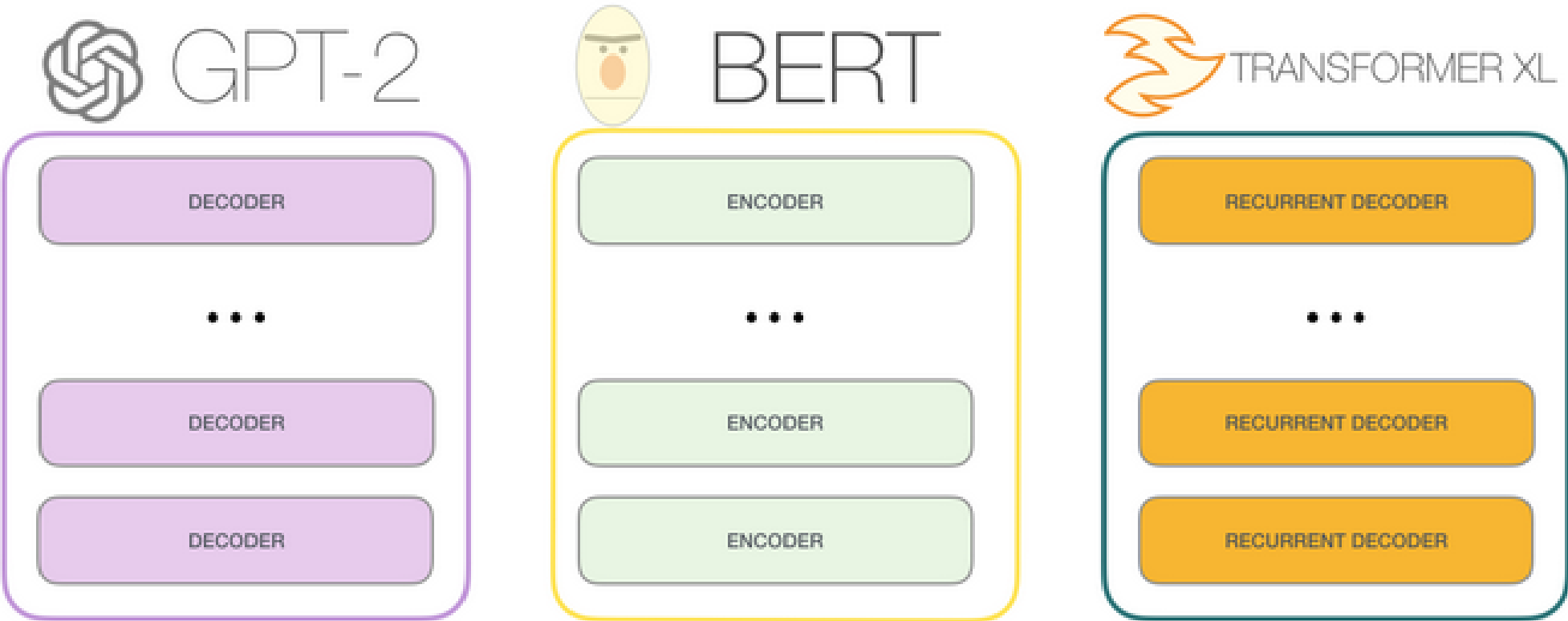
Martin Popel, Ondřej Bojar «Training Tips for the Transformer Model» // <https://arxiv.org/pdf/1804.00247.pdf>

**визуализация обучения:**

<https://ai.googleblog.com/2017/08/transformer-novel-neural-network.html>

Использование трансформера

encoder-only (классификация)  
decoder-only (LM)  
encoder-decoder (перевод)





**BERT и ELMo**

**(Devlin et al, 2018)**



**(Peters et al, 2018)**

## **BERT = Bidirectional Encoder Representations from Transformers**

**transformer network для предобучения модели языка и получения представления слов**  
– идея трансферного обучения

### **Идея из GPT:**

**давайте возьмём LM – обучим на большом корпусе,  
будем «поднастраивать» на конкретные задачи**

### **Фишки:**

- **двунаправленность (точнее, полный обзор) при обучении**
- **разные задачи для преднастройки (в отличие от ELMo и OpenAI-GPT)**  
маскирование, предсказание следующего предложения

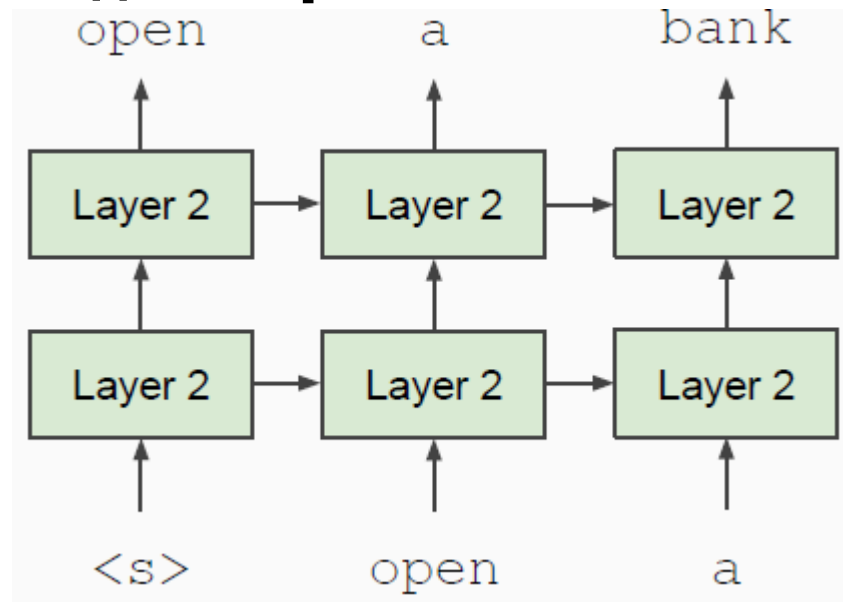
**J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova «Bert: Pre-training of deep bidirectional transformers for language understanding». 2018. <https://arxiv.org/abs/1810.04805>**



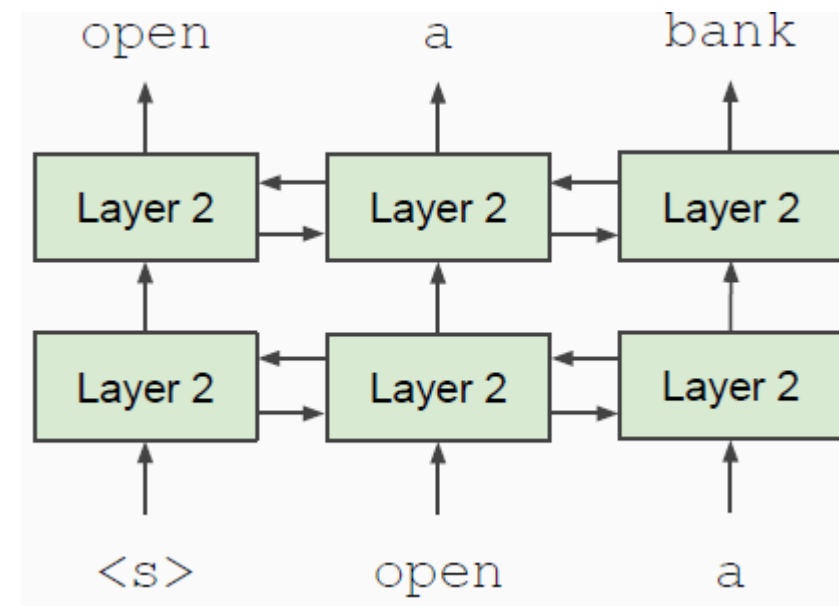
## BERT: учёт контекста

**обычно левый или правый контекст, а надо «двусторонний»**  
**главный подводный камень – слова увидят себя**

### односторонний контекст



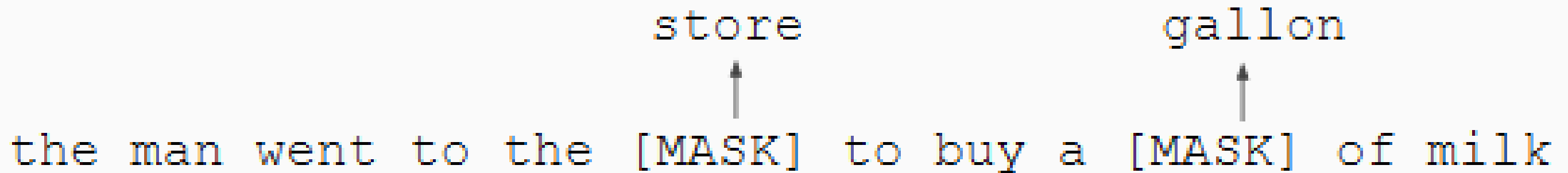
### двусторонний контекст



**слова видят друг друга**

## BERT: Mask language model (MLM)

**решение: маскировать  $k\%$  слов и предсказывать их**



store                      gallon

↑                                      ↑

the man went to the [MASK] to buy a [MASK] of milk

**15% случайно выбранных токенов маскируем – их предсказываем**

**$p=0.8$  – заменять на [MASK]**

went to the store → went to the [MASK]

**$p=0.1$  – заменять на случайное слово**

went to the store → went to the running

**$p=0.1$  – оставлять**

went to the store → went to the store

## BERT: Связь между предложениями (Next sentence prediction)

**Решаем такую задачу: предложение В после А или нет**  
**т.е. это не совсем, как в дословном переводе «предсказание»**

**Sentence A** = The man went to the store.  
**Sentence B** = He bought a gallon of milk.  
**Label** = IsNextSentence

**Sentence A** = The man went to the store.  
**Sentence B** = Penguins are flightless.  
**Label** = NotNextSentence

**генерация выборки: соотношение классов 1/0 = 50/50**

BERT: Представление входа (Input Representation)

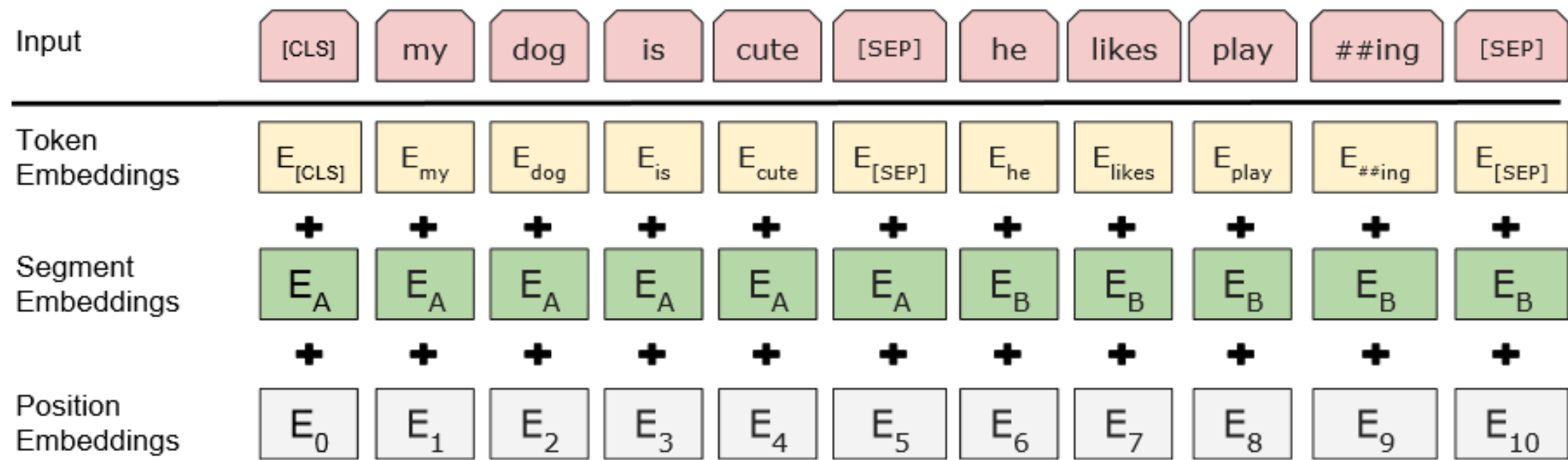


Figure 2: BERT input representation. The input embeddings are the sum of the token embeddings, the segmentation embeddings and the position embeddings.

**каждый токен = сумма 3х вложений**  
**2 предложения на входе разделяются спецсимволом**  
**Представление позиции – обучаемо**

## BERT: Детали

- **Data: Wikipedia (2.5B words) + BookCorpus (800M words)**
- **Batch Size: 131,072 words (1024 sequences \* 128 length or 256 sequences \* 512 length)**
- **Training Time: 1M steps (~40 epochs)**
- **Optimizer: AdamW, 1e-4 learning rate, linear decay (после 10000 шагов к исходному состоянию)**
- **dropout = 0.1 на всех слоях**
- **BERT-Base: 12-layer, 768-hidden, 12-head (базовая архитектура)**
- **BERT-Large: 24-layer, 1024-hidden, 16-head (большая архитектура)**
- **Trained on 4x4 or 8x8 TPU slice for 4 days**
- **WordPiece-токенизация (30 000 токенов)**

# BERT vs GPT vs ELMo

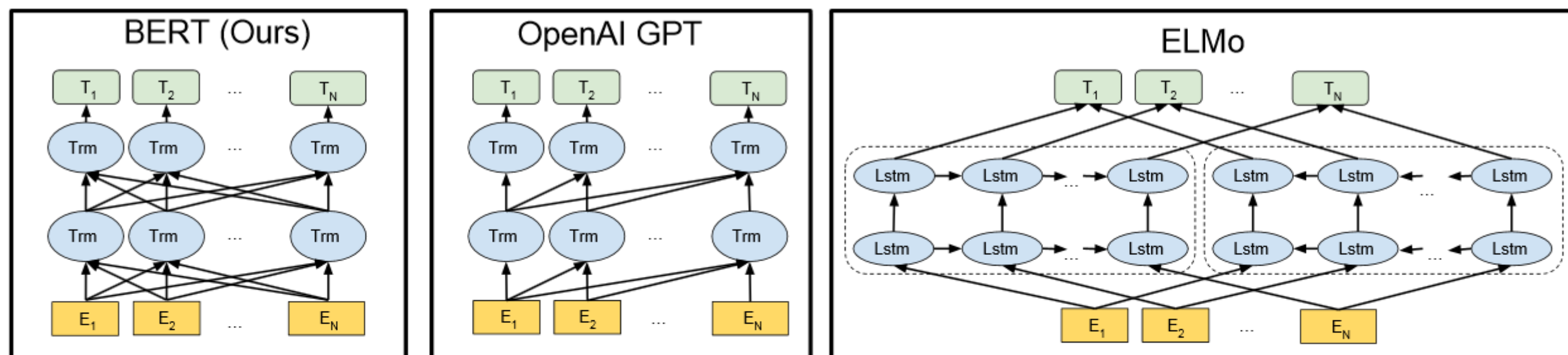


Figure 3: Differences in pre-training model architectures. BERT uses a bidirectional Transformer. OpenAI GPT uses a left-to-right Transformer. ELMo uses the concatenation of independently trained left-to-right and right-to-left LSTMs to generate features for downstream tasks. Among the three, only BERT representations are jointly conditioned on both left and right context in all layers. In addition to the architecture differences, BERT and OpenAI GPT are fine-tuning approaches, while ELMo is a feature-based approach.

## BERT: схема обучения и дообучения на конкретную задачу

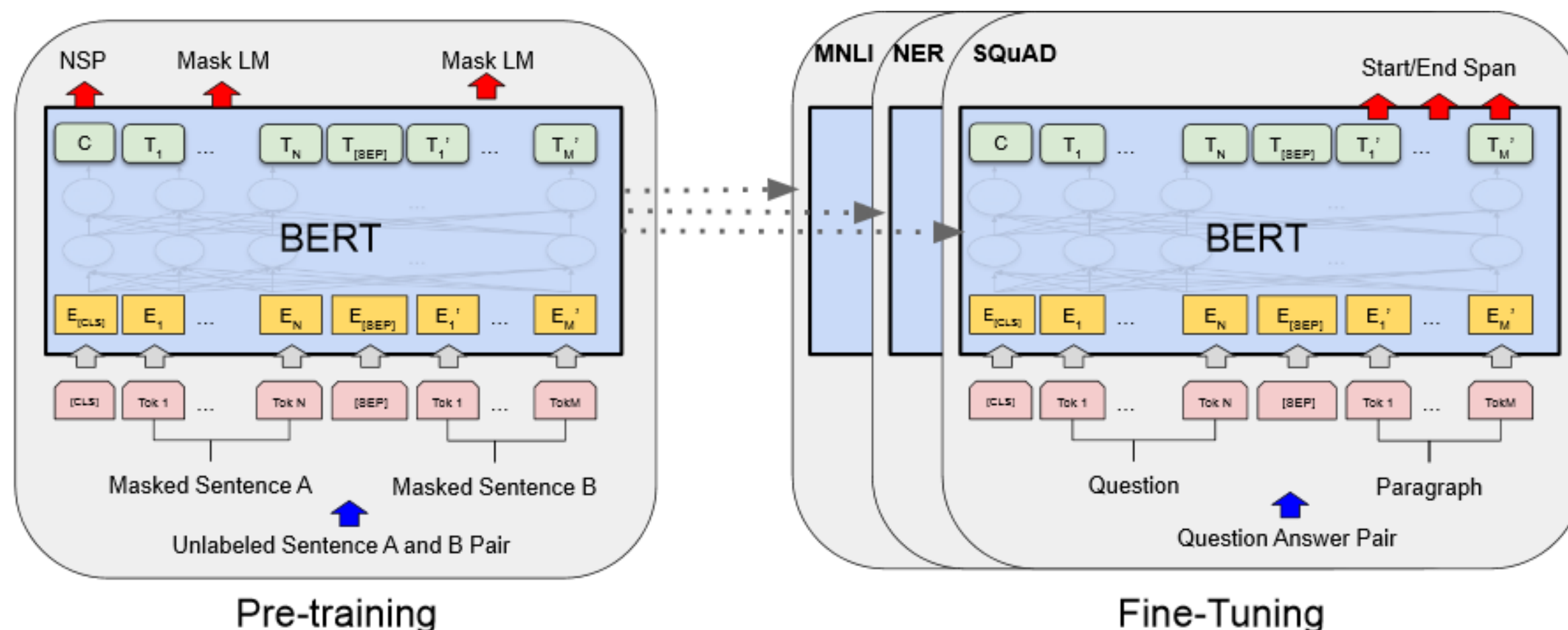
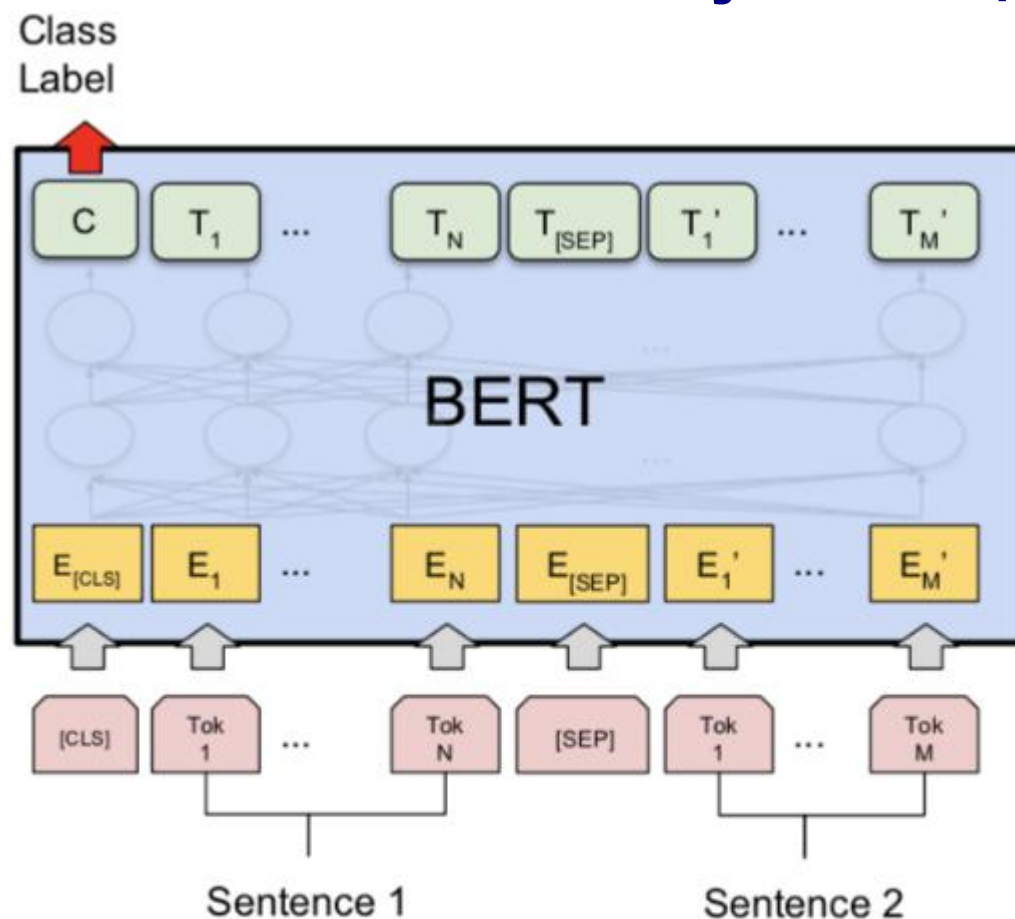
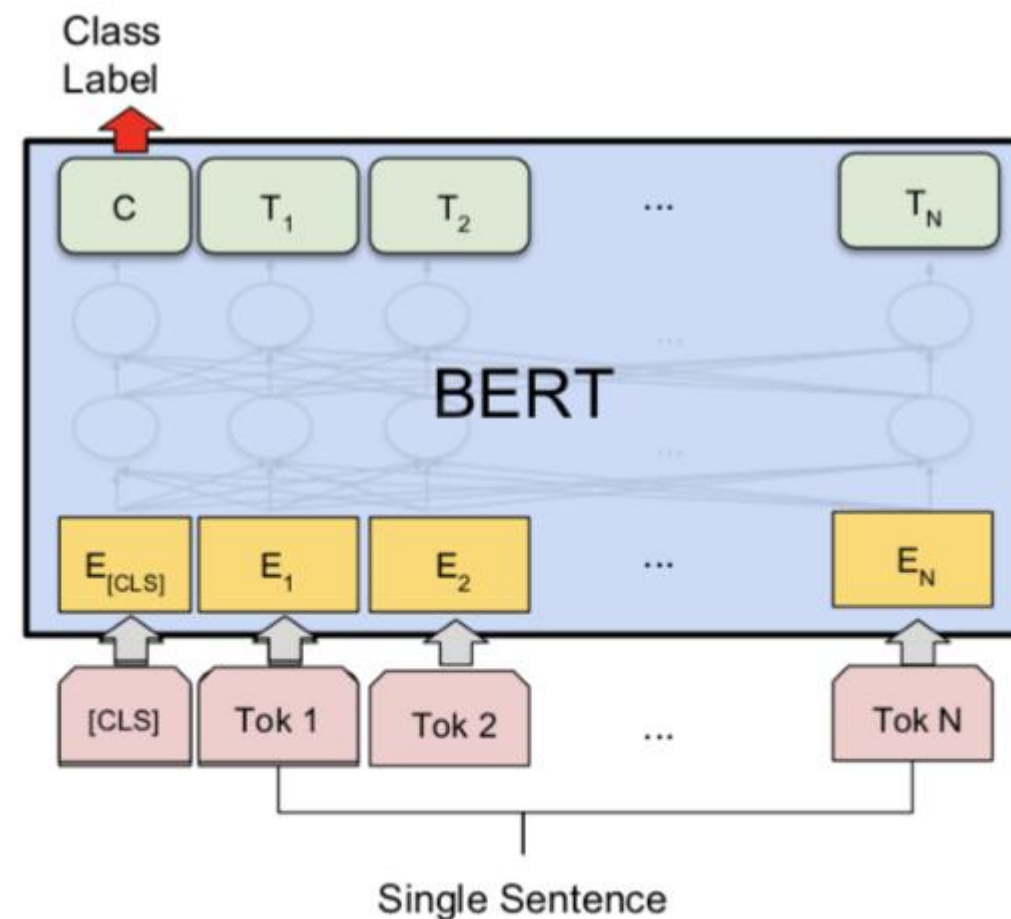


Figure 1: Overall pre-training and fine-tuning procedures for BERT. Apart from output layers, the same architectures are used in both pre-training and fine-tuning. The same pre-trained model parameters are used to initialize models for different down-stream tasks. During fine-tuning, all parameters are fine-tuned. [CLS] is a special symbol added in front of every input example, and [SEP] is a special separator token (e.g. separating questions/answers).



**BERT: схема обучения и дообучения на конкретную задачу**

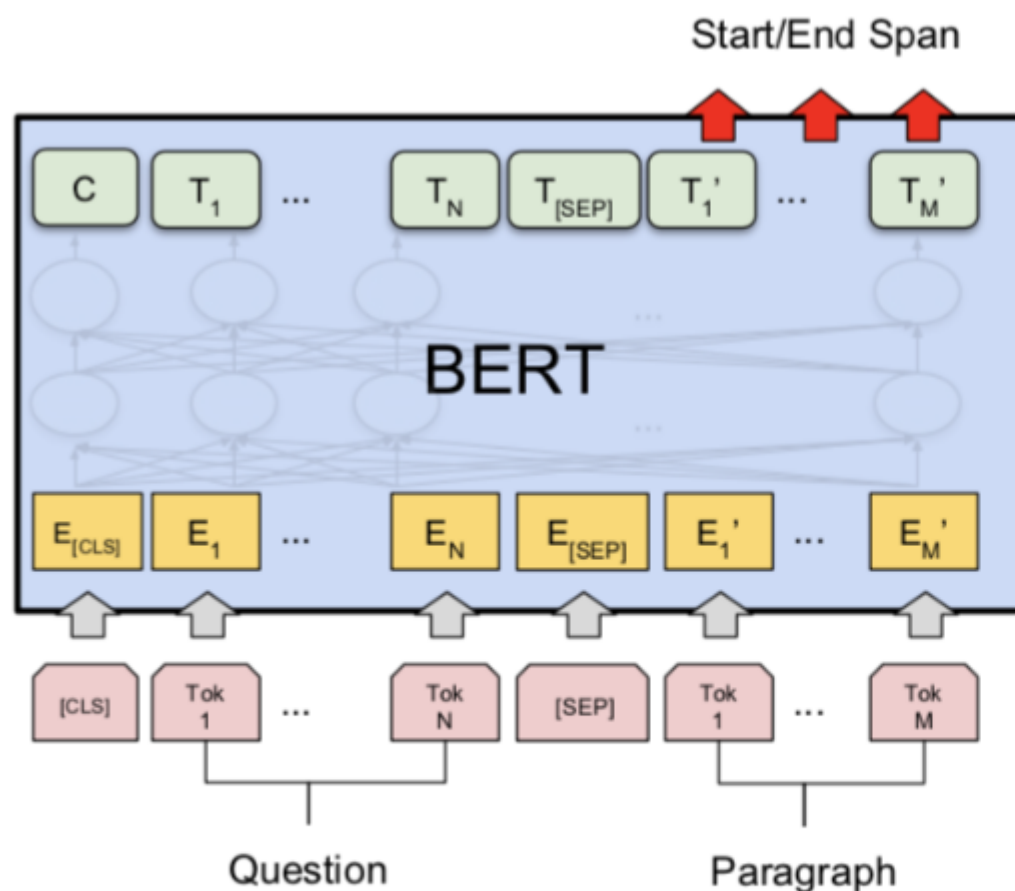
(a) Sentence Pair Classification Tasks:  
MNLI, QQP, QNLI, STS-B, MRPC,  
RTE, SWAG



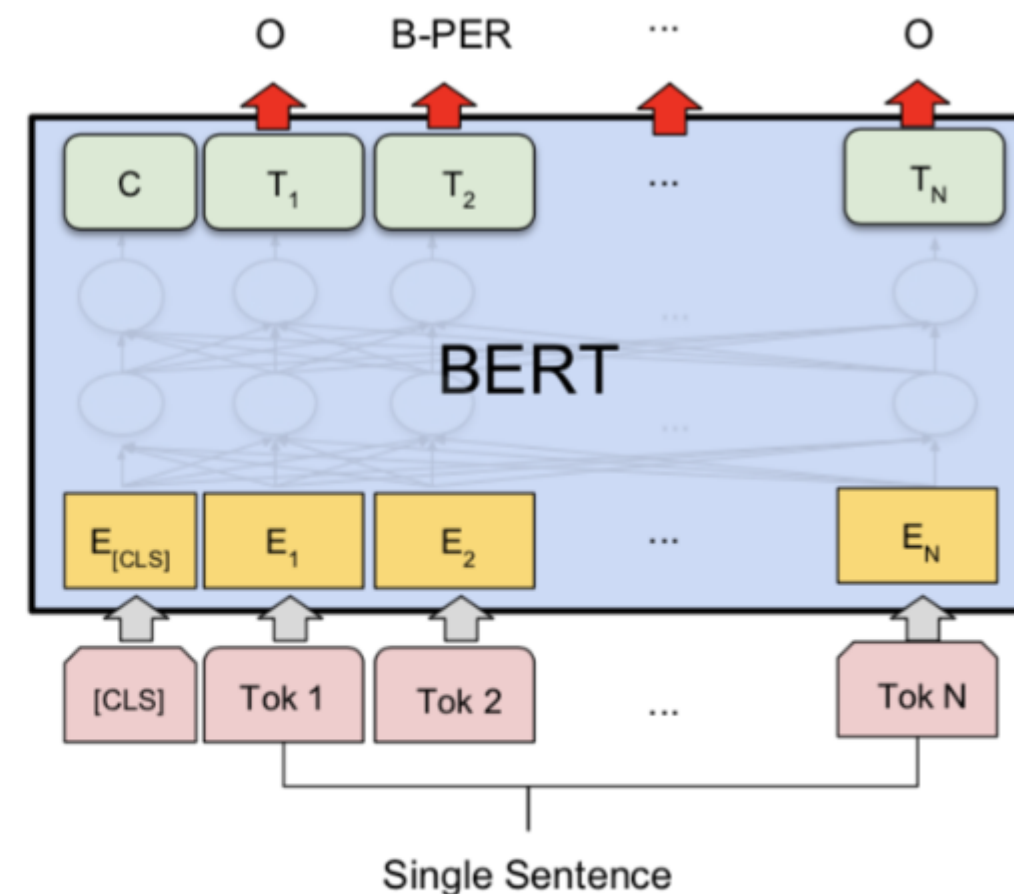
(b) Single Sentence Classification Tasks:  
SST-2, CoLA

**для классификации берём финальное состояние специального первого токена [CLS]  
умножаем его на обучаемую матрицу и берём softmax**



**BERT: схема обучения и дообучения на конкретную задачу**

(c) Question Answering Tasks:  
SQuAD v1.1

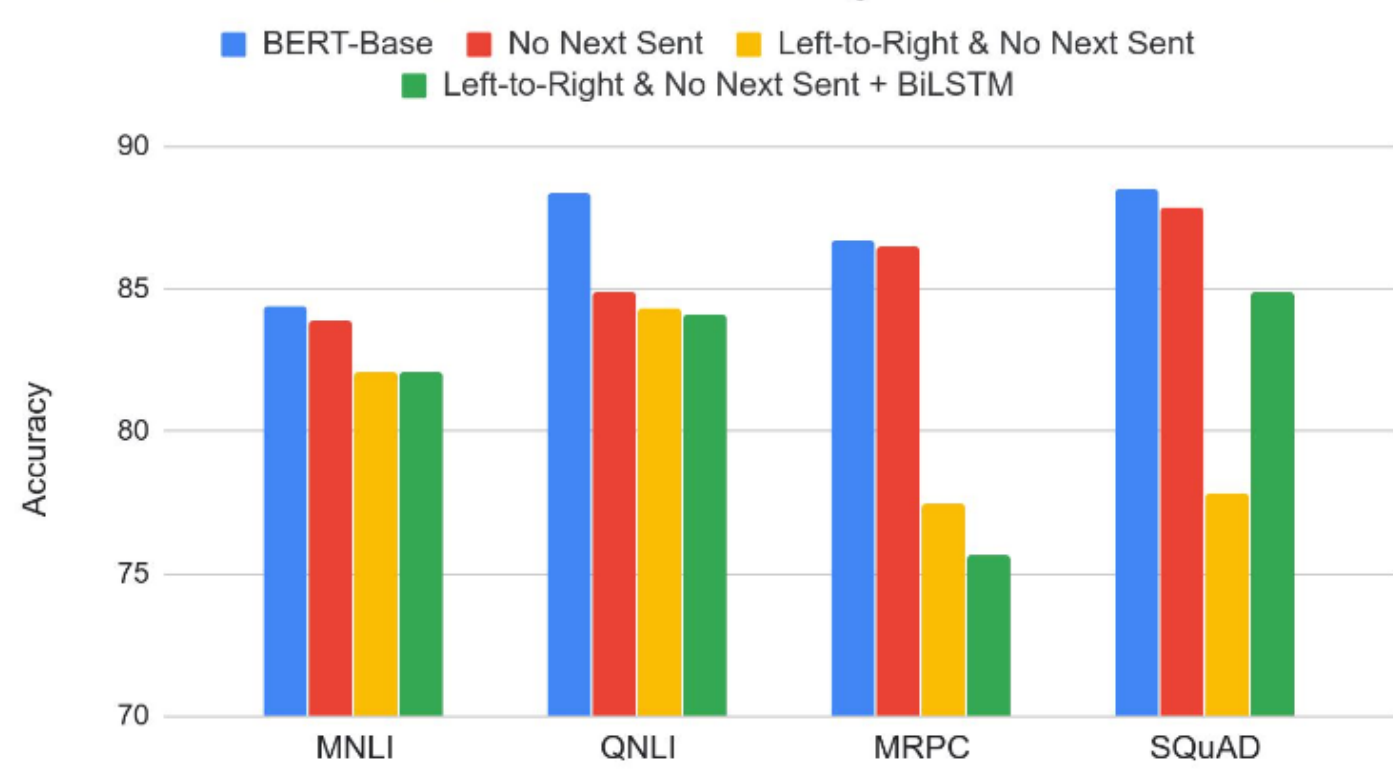


(d) Single Sentence Tagging Tasks:  
CoNLL-2003 NER

**QA: ответ «кусочек текста» – для каждого токена предсказываем вероятность быть началом и концом, обучаемые параметры – две матрицы, которые умножаются на состояния с последующим softmax, чтобы получить вероятности**

## BERT: Эффект предобучения

Effect of Pre-training Task



| Tasks                | Dev Set         |               |               |                |               |
|----------------------|-----------------|---------------|---------------|----------------|---------------|
|                      | MNLI-m<br>(Acc) | QNLI<br>(Acc) | MRPC<br>(Acc) | SST-2<br>(Acc) | SQuAD<br>(F1) |
| BERT <sub>BASE</sub> | 84.4            | 88.4          | 86.7          | 92.7           | 88.5          |
| No NSP               | 83.9            | 84.9          | 86.5          | 92.6           | 87.9          |
| LTR & No NSP         | 82.1            | 84.3          | 77.5          | 92.1           | 77.8          |
| + BiLSTM             | 82.1            | 84.1          | 75.7          | 91.6           | 84.9          |

Table 5: Ablation over the pre-training tasks using the BERT<sub>BASE</sub> architecture. “No NSP” is trained without the next sentence prediction task. “LTR & No NSP” is trained as a left-to-right LM without the next sentence prediction, like OpenAI GPT. “+ BiLSTM” adds a randomly initialized BiLSTM on top of the “LTR + No NSP” model during fine-tuning.

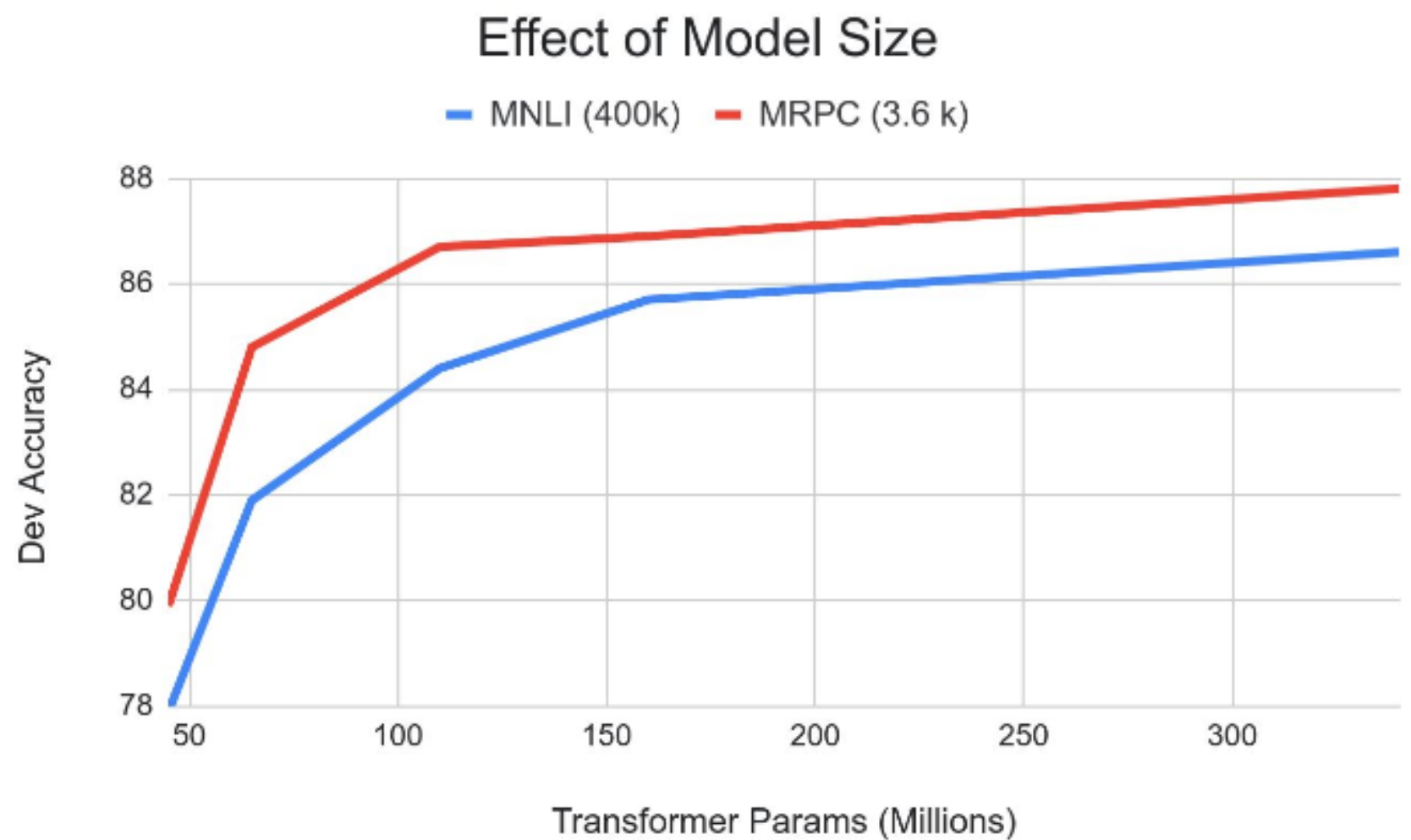
- Для каких-то задач важнее Masked LM, для каких-то NSP
  - на SQuAD плохи модели «слева-направо»

**BERT: результаты**

| System                | MNLI-(m/mm)<br>392k | QQP<br>363k | QNLI<br>108k | SST-2<br>67k | CoLA<br>8.5k | STS-B<br>5.7k | MRPC<br>3.5k | RTE<br>2.5k | Average<br>- |
|-----------------------|---------------------|-------------|--------------|--------------|--------------|---------------|--------------|-------------|--------------|
| Pre-OpenAI SOTA       | 80.6/80.1           | 66.1        | 82.3         | 93.2         | 35.0         | 81.0          | 86.0         | 61.7        | 74.0         |
| BiLSTM+ELMo+Attn      | 76.4/76.1           | 64.8        | 79.8         | 90.4         | 36.0         | 73.3          | 84.9         | 56.8        | 71.0         |
| OpenAI GPT            | 82.1/81.4           | 70.3        | 87.4         | 91.3         | 45.4         | 80.0          | 82.3         | 56.0        | 75.1         |
| BERT <sub>BASE</sub>  | 84.6/83.4           | 71.2        | 90.5         | 93.5         | 52.1         | 85.8          | 88.9         | 66.4        | 79.6         |
| BERT <sub>LARGE</sub> | <b>86.7/85.9</b>    | <b>72.1</b> | <b>92.7</b>  | <b>94.9</b>  | <b>60.5</b>  | <b>86.5</b>   | <b>89.3</b>  | <b>70.1</b> | <b>82.1</b>  |

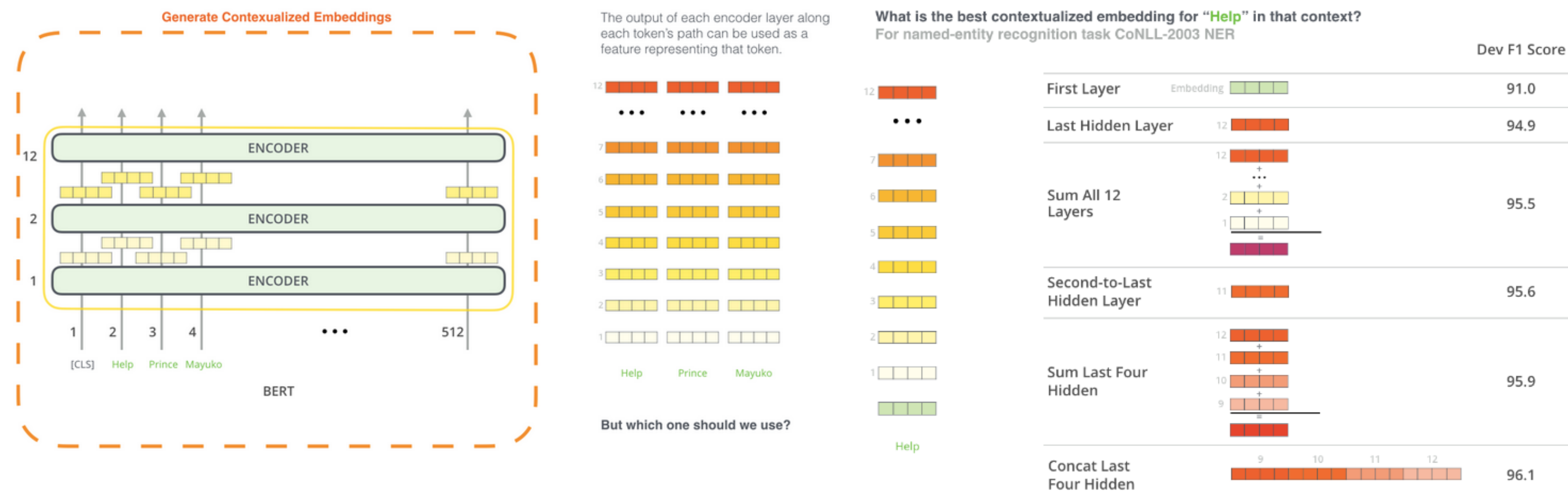
Table 1: GLUE Test results, scored by the evaluation server (<https://gluebenchmark.com/leaderboard>). The number below each task denotes the number of training examples. The “Average” column is slightly different than the official GLUE score, since we exclude the problematic WNLI set.<sup>8</sup> BERT and OpenAI GPT are single-model, single task. F1 scores are reported for QQP and MRPC, Spearman correlations are reported for STS-B, and accuracy scores are reported for the other tasks. We exclude entries that use BERT as one of their components.

BERT: эффект размера модели



**глубина имеет значение (даже для относительно небольших датасетов)  
не вышли на асимптоту;)**

Представления слов с помощью BERT



как в EIMo можно комбинировать состояния разных уровней

## Итог

**Трансформер – оригинальная нерекуррентная архитектура**  
**Сейчас почти все лучшие решения основаны на трансформере**

### Тренды:

обучение на большом корпусе  
подстраивание под конкретные задачи  
«упрощение»: уменьшение числа параметров, слоёв

### Центральная идея современного NLP – «Transfer Learning»

предобучаемся на задаче с большими данными и дешёвой разметкой  
переносим модель и дообучаем под нашу задачу

### Пример аннотированного кода

<http://nlp.seas.harvard.edu/2018/04/03/attention.html>